3. Practice SQL Data Definition Language (DDL) Commands

3.1. Table creation and alteration

1. Create all 5 tables based on the schema provided.

Q: Table Name: Departments **Description:** Stores Department Information

Column Name	Data Type	Size	Attributes
Dept_ID	VARCHAR2	4	Primary Key, Must start with 'D'
Dept_Name	VARCHAR2	50	Not Null, Unique
Building	VARCHAR2	30	—
Number_of_Classrooms	NUMBER	3	CHECK (Number_of_Classrooms >= 0)

CODE:

```
CREATE TABLE Departments (
   Dept_ID VARCHAR2(4) PRIMARY KEY CHECK (Dept_ID LIKE 'D%'),
   Dept_Name VARCHAR2(50) NOT NULL UNIQUE,
   Building VARCHAR2(30),
   Number_of_Classrooms NUMBER(3) CHECK (Number_of_Classrooms >= 0)
);
```

OUTPUT:

```
SQL> CREATE TABLE Departments (
  2      Dept_ID VARCHAR2(4) PRIMARY KEY CHECK (Dept_ID LIKE 'D%'),
  3      Dept_Name VARCHAR2(50) NOT NULL UNIQUE,
  4      Building VARCHAR2(30),
  5      Number_of_Classrooms NUMBER(3) CHECK (Number_of_Classrooms >= 0)
  6  );

Table created.
```

Q: Table Name: Professors **Description:** Stores Professor Information

Column Name	Data Type	Size	Attributes
Prof_ID	VARCHAR2	5	Primary Key, Must start with 'P'
Prof_Name	VARCHAR2	50	Not Null
Dept_ID	VARCHAR2	4	Foreign Key → Departments(Dept_ID)
Experience_Years	NUMBER	3	CHECK (Experience_Years ≥ 0)

CODE:

```
CREATE TABLE Professors (
   Prof_ID VARCHAR2(5) PRIMARY KEY CHECK (Prof_ID LIKE 'P%'),
   Prof_Name VARCHAR2(50) NOT NULL,
   Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
   Experience_Years NUMBER(2) CHECK (Experience_Years >= 0)
);
```

OUTPUT:

```
SQL> CREATE TABLE Professors (
  2      Prof_ID VARCHAR2(5) PRIMARY KEY CHECK (Prof_ID LIKE 'P%'),
  3      Prof_Name VARCHAR2(50) NOT NULL,
  4      Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
  5      Experience_Years NUMBER(2) CHECK (Experience_Years >= 0)
  6  );

Table created.
```

Q: Table Name: Courses **Description:** Stores Course Information

Column Name	Data Type	Size	Attributes
Course_ID	VARCHAR2	6	Primary Key
Course_Name	VARCHAR2	100	Not Null
Dept_ID	VARCHAR2	4	Foreign Key → Departments(Dept_ID)
Prof_ID	VARCHAR2	5	Foreign Key → Professors(Prof_ID)
Credits	NUMBER	1	CHECK (Credits BETWEEN 1 AND 5)
Total_Students	NUMBER	4	CHECK (Total_Students ≥ 0)

CODE:

```
CREATE TABLE Courses (
   Course_ID VARCHAR2(6) PRIMARY KEY,
   Course_Name VARCHAR2(100) NOT NULL,
   Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
   Prof_ID VARCHAR2(5) REFERENCES Professors(Prof_ID),
   Credits NUMBER(1) CHECK (Credits BETWEEN 1 AND 5),
   Student_Count NUMBER(4) CHECK (Student_Count >= 0)
);
```

OUTPUT:

```
SQL> CREATE TABLE Courses (
  2      Course_ID VARCHAR2(6) PRIMARY KEY,
  3      Course_Name VARCHAR2(100) NOT NULL,
  4      Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
  5      Prof_ID VARCHAR2(5) REFERENCES Professors(Prof_ID),
  6      Credits NUMBER(1) CHECK (Credits BETWEEN 1 AND 5),
  7      Student_Count NUMBER(4) CHECK (Student_Count >= 0)
  8  );

Table created.
```

Q: Table Name: Students **Description:** Stores Student Information

Column Name	Data Type	Size	Attributes
Student_ID	VARCHAR2	6	Primary Key, Must start with 'S'
Student_Name	VARCHAR2	50	Not Null
Dept_ID	VARCHAR2	4	Foreign Key → Departments(Dept_ID)
DOB	DATE	—	—

CODE:

```
CREATE TABLE Students (
   Student_ID VARCHAR2(6) PRIMARY KEY CHECK (Student_ID LIKE 'S%'),
   Student_Name VARCHAR2(50) NOT NULL,
   Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
   DOB DATE,
);
```

OUTPUT:

```
SQL> CREATE TABLE Students (
  2      Student_ID VARCHAR2(6) PRIMARY KEY CHECK (Student_ID LIKE 'S%'),
  3      Student_Name VARCHAR2(50) NOT NULL,
  4      Dept_ID VARCHAR2(4) REFERENCES Departments(Dept_ID),
  5      DOB DATE
  6  );

Table created.
```

Q: Table Name: Enrollments **Description:** Stores Student Enrollments in Courses

Column Name	Data Type	Size	Attributes
Student_ID	VARCHAR2	6	Part of Primary Key, Foreign Key → Students(Student_ID)
Course_ID	VARCHAR2	6	Part of Primary Key, Foreign Key → Courses(Course_ID)
Semester	VARCHAR2	6	Format e.g., 'Sem1', 'Sem2'
Marks	NUMBER	5,2	CHECK (Marks ≥ 0 AND Marks < 100)

CODE:

```
CREATE TABLE Enrollments (
   Student_ID VARCHAR2(6) REFERENCES Students(Student_ID),
   Course_ID VARCHAR2(6) REFERENCES Courses(Course_ID),
   Semester VARCHAR2(6),
   Marks NUMBER(5,2) CHECK (Marks >= 0 AND Marks < 100),
   PRIMARY KEY (Student_ID, Course_ID)
);
```

OUTPUT:

```
SQL> CREATE TABLE Enrollments (
  2      Student_ID VARCHAR2(6) REFERENCES Students(Student_ID),
  3      Course_ID VARCHAR2(6) REFERENCES Courses(Course_ID),
  4      Semester VARCHAR2(6),
  5      Marks NUMBER(5,2) CHECK (Marks >= 0 AND Marks < 100),
  6      PRIMARY KEY (Student_ID, Course_ID)
  7  );

Table created.
```

2. Add a column Dept_Head (varchar2(50)) to Departments.

Q: Table Name: Departments **Description:** Stores Department Information

Column Name	Data Type	Size	Attributes
Dept_ID	VARCHAR2	4	Primary Key, Must start with 'D'
Dept_Name	VARCHAR2	50	Not Null, Unique
Building	VARCHAR2	30	—
Number_of_Classrooms	NUMBER	3	CHECK (Number_of_Classrooms >= 0)
Dept_Head	VARCHAR2	50	—

CODE:

```
ALTER TABLE Departments
ADD Dept_Head VARCHAR2(50);
```

OUTPUT:

```
SQL> ALTER TABLE Departments
  2  ADD Dept_Head VARCHAR2(50);

Table altered.
```

3. Change size of Experience_Years in Professors to NUMBER(3).

Q: Table Name: Professors **Description:** Stores Professor Information

Column Name	Data Type	Size	Attributes
Prof_ID	VARCHAR2	5	Primary Key, Must start with 'P'
Prof_Name	VARCHAR2	50	Not Null
Dept_ID	VARCHAR2	4	Foreign Key → Departments(Dept_ID)
Experience_Years	NUMBER	3	CHECK (Experience_Years ≥ 0)

CODE:

```
ALTER TABLE Professors
MODIFY Experience_Years NUMBER(3);
```

OUTPUT:

```
SQL> ALTER TABLE Professors
  2  MODIFY Experience_Years NUMBER(3);

Table altered.
```

4. Rename Student_Count to Total_Students in Courses.

Q: Table Name: Courses **Description:** Stores Course Information

Column Name	Data Type	Size	Attributes
Course_ID	VARCHAR2	6	Primary Key
Course_Name	VARCHAR2	100	Not Null
Dept_ID	VARCHAR2	4	Foreign Key → Departments(Dept_ID)
Prof_ID	VARCHAR2	5	Foreign Key → Professors(Prof_ID)
Credits	NUMBER	1	CHECK (Credits BETWEEN 1 AND 5)
Total_Students	NUMBER	4	CHECK (Total_Students ≥ 0)

CODE:

```
ALTER TABLE Courses
RENAME COLUMN Student_Count TO Total_Students;
```

OUTPUT:

```
SQL> ALTER TABLE Courses
  2  RENAME COLUMN Student_Count TO Total_Students;

Table altered.
```

5. Drop and recreate the Enrollments table.

Q: Table Name: Enrollments **Description:** Stores Student Enrollments in Courses

Column Name	Data Type	Size	Attributes
Student_ID	VARCHAR2	6	Part of Primary Key, Foreign Key → Students(Student_ID)
Course_ID	VARCHAR2	6	Part of Primary Key, Foreign Key → Courses(Course_ID)
Semester	VARCHAR2	6	Format e.g., 'Sem1', 'Sem2'
Marks	NUMBER	5,2	CHECK (Marks ≥ 0 AND Marks < 100)

CODE:

a) Drop the table

```
DROP TABLE Enrollments;
```

b) Recreate the table

```
CREATE TABLE Enrollments (
   Student_ID VARCHAR2(6) REFERENCES Students(Student_ID),
   Course_ID VARCHAR2(6) REFERENCES Courses(Course_ID),
   Semester VARCHAR2(6),
   Marks NUMBER(5,2) CHECK (Marks >= 0 AND Marks < 100),
   PRIMARY KEY (Student_ID, Course_ID)
);
```

OUTPUT:

```
SQL> DROP TABLE Enrollments;

Table dropped.

SQL> CREATE TABLE Enrollments (
  2      Student_ID VARCHAR2(6) REFERENCES Students(Student_ID),
  3      Course_ID VARCHAR2(6) REFERENCES Courses(Course_ID),
  4      Semester VARCHAR2(6),
  5      Marks NUMBER(5,2) CHECK (Marks >= 0 AND Marks < 100),
  6      PRIMARY KEY (Student_ID, Course_ID)
  7  );

Table created.
```

4. Practice SQL Data Manipulation Language (DML) Commands

4.1. Insertion, Deletion, Update

1. Insert the sample data into all five tables.

Table Name: Departments

Dept_ID	Dept_Name	Location	No_of_Staff
D01	Computer Science	Tech Block	10
D02	Electrical Engg.	Power House	8
D03	Mechanical Engg.	Mech Block	6

CODE:

```
INSERT INTO Departments VALUES ('D01', 'Computer Science', 'Tech Block', 10);
INSERT INTO Departments VALUES ('D02', 'Electrical Engg.', 'Power House', 8);
INSERT INTO Departments VALUES ('D03', 'Mechanical Engg.', 'Mech Block', 6);
```

OUTPUT:

```
SQL> INSERT INTO Departments VALUES
  2  ('D01', 'Computer Science', 'Tech Block', 10);

1 row created.

SQL> INSERT INTO Departments VALUES
  2  ('D02', 'Electrical Engg.', 'Power House', 8);

1 row created.

SQL> INSERT INTO Departments VALUES
  2  ('D03', 'Mechanical Engg.', 'Mech Block', 6);

1 row created.

SQL> SELECT * FROM Departments;

DEPT DEPT_NAME         BUILDING    NUMBER_OF_CLASSROOMS
---- ----------------- ----------- --------------------
D01  Computer Science  Tech Block                   10
D02  Electrical Engg.  Power House                   8
D03  Mechanical Engg.  Mech Block                    6
```

Table Name: Professors

Prof_ID	Prof_Name	Dept_ID	Experience_Years
P1001	Dr. Meera Nair	D01	12
P1002	Dr. Arjun Rao	D02	9
P1003	Dr. Kavita Singh	D01	7
P1004	Dr. Raj Malhotra	D03	15

CODE:

```
INSERT INTO Professors VALUES ('P1001', 'Dr. Meera Nair', 'D01', 12);
INSERT INTO Professors VALUES ('P1002', 'Dr. Arjun Rao', 'D02', 9);
INSERT INTO Professors VALUES ('P1003', 'Dr. Kavita Singh', 'D01', 7);
INSERT INTO Professors VALUES ('P1004', 'Dr. Raj Malhotra', 'D03', 15);
```

OUTPUT:

```
SQL> INSERT INTO Professors VALUES
  2  ('P1001', 'Dr. Meera Nair', 'D01', 12);

1 row created.

SQL> INSERT INTO Professors VALUES
  2  ('P1002', 'Dr. Arjun Rao', 'D02', 9);

1 row created.

SQL> INSERT INTO Professors VALUES
  2  ('P1003', 'Dr. Kavita Singh', 'D01', 7);

1 row created.

SQL> INSERT INTO Professors VALUES
  2  ('P1004', 'Dr. Raj Malhotra', 'D03', 15);

1 row created.

SQL> SELECT * FROM Professors;

PROF_ID PROF_NAME           DEPT_ID EXPERIENCE_YEARS
------- ------------------- ------- ----------------
P1001   Dr. Meera Nair      D01                  12
P1002   Dr. Arjun Rao       D02                   9
P1003   Dr. Kavita Singh    D01                   7
P1004   Dr. Raj Malhotra    D03                  15
```

Table Name: Courses

Course_ID	Course_Name	Dept_ID	Prof_ID	Credits	Total_Students
CSE101	Data Structures	D01	P1001	4	2
CSE201	Operating Systems	D01	P1003	3	1
EEE101	Circuit Theory	D02	P1002	4	1
ME101	Thermodynamics	D03	P1004	3	1

CODE:

```
INSERT INTO Courses VALUES ('CSE101', 'Data Structures', 'D01', 'P1001', 4, 2);
INSERT INTO Courses VALUES ('CSE201', 'Operating Systems', 'D01', 'P1003', 3, 1);
INSERT INTO Courses VALUES ('EEE101', 'Circuit Theory', 'D02', 'P1002', 4, 1);
INSERT INTO Courses VALUES ('ME101', 'Thermodynamics', 'D03', 'P1004', 3, 1);
```

OUTPUT:

```
SQL> INSERT INTO Courses VALUES
  2  ('CSE101', 'Data Structures', 'D01', 'P1001', 4, 2);

1 row created.

SQL> INSERT INTO Courses VALUES
  2  ('CSE201', 'Operating Systems', 'D01', 'P1003', 3, 1);

1 row created.

SQL> INSERT INTO Courses VALUES
  2  ('EEE101', 'Circuit Theory', 'D02', 'P1002', 4, 1);

1 row created.

SQL> INSERT INTO Courses VALUES
  2  ('ME101', 'Thermodynamics', 'D03', 'P1004', 3, 1);

1 row created.

SQL> SELECT * FROM Courses;

COURSE_ID COURSE_NAME          DEPT_ID PROF_ID    CREDITS TOTAL_STUDENTS
--------- -------------------- ------- ------- ---------- --------------
CSE101    Data Structures      D01     P1001            4              2
CSE201    Operating Systems    D01     P1003            3              1
EEE101    Circuit Theory       D02     P1002            4              1
ME101     Thermodynamics       D03     P1004            3              1
```

Table Name: Students

Student_ID	Student_Name	Dept_ID	DOB
S0001	Anjali Sharma	D01	2003-05-14
S0002	Ravi Kumar	D02	2002-11-20
S0003	Nisha Verma	D03	2003-02-02
S0004	Aman Sheikh	D01	2002-07-25

CODE:

```
INSERT INTO Students VALUES
('S0001', 'Anjali Sharma', 'D01', TO_DATE('2003-05-14', 'YYYY-MM-DD'));
INSERT INTO Students VALUES
('S0002', 'Ravi Kumar', 'D02', TO_DATE('2002-11-20', 'YYYY-MM-DD'));
INSERT INTO Students VALUES
('S0003', 'Nisha Verma', 'D03', TO_DATE('2003-02-02', 'YYYY-MM-DD'));
INSERT INTO Students VALUES
('S0004', 'Aman Sheikh', 'D01', TO_DATE('2002-07-25', 'YYYY-MM-DD'));
```

OUTPUT:

```
SQL> INSERT INTO Students VALUES
  2  ('S0001', 'Anjali Sharma', 'D01', TO_DATE('2003-05-14', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Students VALUES
  2  ('S0002', 'Ravi Kumar', 'D02', TO_DATE('2002-11-20', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Students VALUES
  2  ('S0003', 'Nisha Verma', 'D03', TO_DATE('2003-02-02', 'YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Students VALUES
  2  ('S0004', 'Aman Sheikh', 'D01', TO_DATE('2002-07-25', 'YYYY-MM-DD'));

1 row created.

SQL> SELECT * FROM Students;

STUDENT_ID STUDENT_NAME         DEPT_ID DOB
---------- -------------------- ------- ---------
S0001      Anjali Sharma        D01     14-MAY-03
S0002      Ravi Kumar           D02     20-NOV-02
S0003      Nisha Verma          D03     02-FEB-03
S0004      Aman Sheikh          D01     25-JUL-02
```

Table Name: Enrollments

Student_ID	Course_ID	Semester	Marks
S0001	CSE101	Sem1	88.0
S0001	CSE201	Sem2	76.5
S0002	EEE101	Sem1	81.0
S0003	ME101	Sem1	93.0
S0004	CSE101	Sem1	68.5

CODE:

```
INSERT INTO Enrollments VALUES ('S0001', 'CSE101', 'Sem1', 88.0);
INSERT INTO Enrollments VALUES ('S0001', 'CSE201', 'Sem2', 76.5);
INSERT INTO Enrollments VALUES ('S0002', 'EEE101', 'Sem1', 81.0);
INSERT INTO Enrollments VALUES ('S0003', 'ME101', 'Sem1', 93.0);
INSERT INTO Enrollments VALUES ('S0004', 'CSE101', 'Sem1', 68.5);
```

OUTPUT:

```
SQL> INSERT INTO Enrollments VALUES ('S0001', 'CSE101', 'Sem1', 88.0);

1 row created.

SQL> INSERT INTO Enrollments VALUES ('S0001', 'CSE201', 'Sem2', 76.5);

1 row created.

SQL> INSERT INTO Enrollments VALUES ('S0002', 'EEE101', 'Sem1', 81.0);

1 row created.

SQL> INSERT INTO Enrollments VALUES ('S0003', 'ME101', 'Sem1', 93.0);

1 row created.

SQL> INSERT INTO Enrollments VALUES ('S0004', 'CSE101', 'Sem1', 68.5);

1 row created.

SQL> SELECT * FROM Enrollments;

STUDENT_ID COURSE_ID SEMESTER       MARKS
---------- --------- --------- ----------
S0001      CSE101    Sem1              88
S0001      CSE201    Sem2            76.5
S0002      EEE101    Sem1              81
S0003      ME101     Sem1              93
S0004      CSE101    Sem1            68.5
```

2. Insert a new department: (‘D04’, ‘Civil Engg’, ‘Block C’, 5).

Table Name: Departments

Dept_ID	Dept_Name	Location	No_of_Staff
D04	Civil Engg	Block C	5

CODE:

```
INSERT INTO Departments VALUES
('D04', 'Civil Engg', 'Block C', 5);
```

OUTPUT:

```
SQL> INSERT INTO Departments VALUES
  2  ('D04', 'Civil Engg', 'Block C', 5);

1 row created.

SQL> SELECT * FROM Departments;

DEPT_ID DEPT_NAME         BUILDING    NUMBER_OF_CLASSROOMS
------- ----------------- ----------- --------------------
D01     Computer Science  Tech Block                  10
D02     Electrical Engg.  Power House                 8
D03     Mechanical Engg.  Mech Block                  6
D04     Civil Engg        Block C                     5
```

3. Create a new table high_achievers containing students who scored more than 85 in any course.

CODE:

```
CREATE TABLE high_achievers AS
SELECT * FROM Students
WHERE Student_ID IN (
   SELECT Student_ID
   FROM Enrollments
   WHERE Marks > 85
);
```

OUTPUT:

```
SQL> CREATE TABLE high_achievers AS
  2  SELECT * FROM Students
  3  WHERE Student_ID IN (
  4      SELECT Student_ID
  5      FROM Enrollments
  6      WHERE Marks > 85
  7  );

Table created.

SQL> SELECT * FROM high_achievers;

STUDEN STUDENT_NAME                                       DEPT DOB
------ -------------------------------------------------- ---- ---------
S0001  Anjali Sharma                                      D01  14-MAY-03
S0003  Nisha Verma                                        D03  02-FEB-03
```

4. Create a backup table Courses_Backup with all data from Courses and Professors_Backup with all the data from professors.

CODE:

CREATE TABLE Courses_Backup AS

SELECT * FROM Courses;

CREATE TABLE Professors_Backup AS

SELECT * FROM Professors;

OUTPUT:

```
SQL> CREATE TABLE Courses_Backup AS
  2  SELECT * FROM Courses;

Table created.

SQL> CREATE TABLE Professors_Backup AS
  2  SELECT * FROM Professors;

Table created.
```

5. Add a new course ‘CIV101’, ‘Structural Analysis’, under D04, taught by P1001, with 3 credits and 0 students.

CODE:

```
INSERT INTO Courses VALUES
('CIV101', 'Structural Analysis', 'D04', 'P1001', 3, 0);
```

OUTPUT:

```
SQL> INSERT INTO Courses VALUES
  2  ('CIV101', 'Structural Analysis', 'D04', 'P1001', 3, 0);

1 row created.

SQL> SELECT * FROM Courses;

COURSE_ID COURSE_NAME          DEPT_ID PROF_ID    CREDITS TOTAL_STUDENTS
--------- -------------------- ------- ------- ---------- --------------
CSE101    Data Structures      D01     P1001            4              2
CSE201    Operating Systems    D01     P1003            3              1
EEE101    Circuit Theory       D02     P1002            4              1
ME101     Thermodynamics       D03     P1004            3              1
CIV101    Structural Analysis  D04     P1001            3              0
```

6. Update professor of ‘CIV101’ to P1004 and savepoint SP1.

CODE:

```
UPDATE Courses SET Prof_ID = 'P1004'
WHERE Course_ID = 'CIV101';
SAVEPOINT SP1;
```

OUTPUT:

```
SQL> UPDATE Courses
  2  SET Prof_ID = 'P1004'
  3  WHERE Course_ID = 'CIV101';

1 row updated.

SQL> SAVEPOINT SP1;

Savepoint created.

SQL> SELECT * FROM Courses;

COURSE_ID COURSE_NAME          DEPT_ID PROF_ID    CREDITS TOTAL_STUDENTS
--------- -------------------- ------- ------- ---------- --------------
CSE101    Data Structures      D01     P1001            4              2
CSE201    Operating Systems    D01     P1003            3              1
EEE101    Circuit Theory       D02     P1002            4              1
ME101     Thermodynamics       D03     P1004            3              1
CIV101    Structural Analysis  D04     P1004            3              0
```

7. Change credits of ‘CSE201’ to 4 and set savepoint SP2.

CODE:

```
UPDATE Courses
SET Credits = 4
WHERE Course_ID = 'CSE201';

SAVEPOINT SP2;
```

OUTPUT:

```
SQL> UPDATE Courses
  2  SET Credits = 4
  3  WHERE Course_ID = 'CSE201';

1 row updated.

SQL> SAVEPOINT SP2;

Savepoint created.

SQL> SELECT * FROM Courses;

COURSE_ID COURSE_NAME          DEPT_ID PROF_ID    CREDITS TOTAL_STUDENTS
--------- -------------------- ------- ------- ---------- --------------
CSE101    Data Structures      D01     P1001            4              2
CSE201    Operating Systems    D01     P1003            4              1
EEE101    Circuit Theory       D02     P1002            4              1
ME101     Thermodynamics       D03     P1004            3              1
CIV101    Structural Analysis  D04     P1004            3              0
```

8. Delete all courses from Courses_Backup that have less than 4 credits.

CODE:

```
DELETE FROM Courses_Backup
WHERE Credits < 4;
```

OUTPUT:

```
SQL> DELETE FROM Courses_Backup
  2  WHERE Credits < 4;

2 rows deleted.

SQL>
SQL> SELECT * FROM Courses_Backup;

COURSE_ID COURSE_NAME          DEPT_ID PROF_ID    CREDITS TOTAL_STUDENTS
--------- -------------------- ------- ------- ---------- --------------
CSE101    Data Structures      D01     P1001            4              2
EEE101    Circuit Theory       D02     P1002            4              1
```

9. Delete all professors from Professors_Backup with less than 10 years experience.

CODE:

```
DELETE FROM Professors_Backup
WHERE Experience_Years < 10;
```

OUTPUT:

```
SQL> DELETE FROM Professors_Backup
  2  WHERE Experience_Years < 10;

2 rows deleted.

SQL> SELECT * FROM Professors_Backup;

PROF_ID PROF_NAME           DEPT_ID EXPERIENCE_YEARS
------- ------------------- ------- ----------------
P1001   Dr. Meera Nair      D01                  12
P1004   Dr. Raj Malhotra    D03                  15
```

10. Rollback to SP1 and rename Courses_Backup to Course_Master.

CODE:

```
ROLLBACK TO SP1;

ALTER TABLE Courses_Backup RENAME TO Course_Master;
```

OUTPUT:

```
SQL> ROLLBACK TO SP1;

Rollback complete.

SQL> ALTER TABLE Courses_Backup RENAME TO Course_Master;

Table altered.

SQL> DESC Course_Master;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------
 COURSE_ID                                          VARCHAR2(6)
 COURSE_NAME                               NOT NULL VARCHAR2(100)
 DEPT_ID                                            VARCHAR2(4)
 PROF_ID                                            VARCHAR2(5)
 CREDITS                                            NUMBER(1)
 TOTAL_STUDENTS                                     NUMBER(4)
```

4.2. Data Retrieval (SELECT)

1. List all department names.

CODE:

```
SELECT Dept_Name FROM Departments;
```

OUTPUT:

```
SQL> SELECT Dept_Name FROM Departments;

DEPT_NAME
--------------------------------------------------
Civil Engg
Computer Science
Electrical Engg.
Mechanical Engg.
```

2. Display all data from the Professors table.

CODE:

```
SELECT * FROM Professors;
```

OUTPUT:

```
SQL> SELECT * FROM Professors;

PROF_ID PROF_NAME          DEPT EXPERIENCE_YEARS
------- ------------------ ---- ----------------
P1001   Dr. Meera Nair     D01                12
P1002   Dr. Arjun Rao      D02                 9
P1003   Dr. Kavita Singh   D01                 7
P1004   Dr. Raj Malhotra   D03                15
```

3. List student names and DOBs.

CODE:

SELECT Student_Name, DOB FROM Students;

OUTPUT:

```
SQL> SELECT Student_Name, DOB FROM Students;

STUDENT_NAME                                       DOB
-------------------------------------------------- ---------
Anjali Sharma                                      14-MAY-03
Ravi Kumar                                         20-NOV-02
Nisha Verma                                        02-FEB-03
Aman Sheikh                                        25-JUL-02
```

4. List course names and credits.

CODE:

SELECT Course_Name, Credits FROM Courses;

OUTPUT:

```
SQL> SELECT Course_Name, Credits FROM Courses;

COURSE_NAME            CREDITS
-------------------- ----------
Data Structures               4
Operating Systems             3
Circuit Theory                4
Thermodynamics                3
Structural Analysis           3
```

5. Get courses offered by the 'Computer Science' department.

CODE:

```
SELECT * FROM Courses
WHERE Dept_ID = (
   SELECT Dept_ID FROM Departments
   WHERE Dept_Name = 'Computer Science'
);
```

OUTPUT:

```
SQL> SELECT *
  2  FROM Courses
  3  WHERE Dept_ID = (
  4      SELECT Dept_ID
  5      FROM Departments
  6      WHERE Dept_Name = 'Computer Science'
  7  );

COURSE COURSE_NAME          DEPT PROF_ID    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ------- ---------- --------------
CSE101 Data Structures      D01  P1001            4              2
CSE201 Operating Systems    D01  P1003            3              1
```

6. List professors whose name starts with 'Dr'.

CODE:

```
SELECT * FROM Professors
WHERE Prof_Name LIKE 'Dr%';
```

OUTPUT:

```
SQL> SELECT * FROM Professors
  2  WHERE Prof_Name LIKE 'Dr%';

PROF_ID PROF_NAME          DEPT EXPERIENCE_YEARS
------- ------------------ ---- ----------------
P1001   Dr. Meera Nair     D01                12
P1002   Dr. Arjun Rao      D02                 9
P1003   Dr. Kavita Singh   D01                 7
P1004   Dr. Raj Malhotra   D03                15
```

7. List courses with credits more than 3.

CODE:

```
SELECT * FROM Courses
WHERE Credits > 3;
```

OUTPUT:

```
SQL> SELECT * FROM Courses
  2  WHERE Credits > 3;

COURSE COURSE_NAME          DEPT PROF_ID    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ------- ---------- --------------
CSE101 Data Structures      D01  P1001            4              2
EEE101 Circuit Theory       D02  P1002            4              1
```

8. Display all courses with “Theory” in their name.

CODE:

```
SELECT * FROM Courses
WHERE Course_Name LIKE '%Theory%';
```

OUTPUT:

```
SQL> SELECT * FROM Courses
  2  WHERE Course_Name LIKE '%Theory%';

COURSE COURSE_NAME          DEPT PROF_ID    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ------- ---------- --------------
EEE101 Circuit Theory       D02  P1002            4              1
```

9. List students born after Jan 1, 2003.

CODE:

```
SELECT * FROM Students
WHERE DOB > TO_DATE('2003-01-01', 'YYYY-MM-DD');
```

OUTPUT:

```
SQL> SELECT * FROM Students
  2  WHERE DOB > TO_DATE('2003-01-01', 'YYYY-MM-DD');

STUDEN STUDENT_NAME         DEPT DOB
------ -------------------- ---- ---------
S0001  Anjali Sharma        D01  14-MAY-03
S0003  Nisha Verma          D03  02-FEB-03
```

10. Find all professors with 10+ years experience.

CODE:

SELECT * FROM Professors

WHERE Experience_Years >= 10;

OUTPUT:

```
SQL> SELECT * FROM Professors
  2  WHERE Experience_Years >= 10;

PROF_ID PROF_NAME           DEPT EXPERIENCE_YEARS
------- ------------------- ---- ----------------
P1001   Dr. Meera Nair      D01                12
P1004   Dr. Raj Malhotra    D03                15
```

11. List all courses with more than 1 student.

CODE:

SELECT * FROM Courses

WHERE Total_Students > 1;

OUTPUT:

```
SQL> SELECT * FROM Courses
  2  WHERE Total_Students > 1;

COURSE COURSE_NAME          DEPT PROF_ID    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ------- ---------- -------------
CSE101 Data Structures      D01  P1001            4             2
```

12. Display all distinct semesters from Enrollments.

CODE:

```
SELECT DISTINCT Semester FROM Enrollments;
```

OUTPUT:

```
SQL> SELECT DISTINCT Semester FROM Enrollments;

SEMESTER
----------
Sem1
Sem2
```

13. Show information of students with ID S0001, S0002.

CODE:

```
SELECT * FROM Students
WHERE Student_ID IN ('S0001', 'S0002');
```

OUTPUT:

```
SQL> SELECT * FROM Students
  2  WHERE Student_ID IN ('S0001', 'S0002');

STUDEN STUDENT_NAME         DEPT DOB
------ -------------------- ---- ---------
S0001  Anjali Sharma        D01  14-MAY-03
S0002  Ravi Kumar           D02  20-NOV-02
```

14. Show all courses not in the 'Mechanical Engg' department.

CODE:

```
SELECT * FROM Courses
WHERE Dept_ID <> (
   SELECT Dept_ID FROM Departments
   WHERE Dept_Name = 'Mechanical Engg.'
);
```

OUTPUT:

```
SQL> SELECT * FROM Courses
  2  WHERE Dept_ID <> (
  3      SELECT Dept_ID FROM Departments
  4      WHERE Dept_Name = 'Mechanical Engg.'
  5  );

COURSE COURSE_NAME          DEPT PROF_ID    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ------- ---------- --------------
CSE101 Data Structures      D01  P1001            4              2
CSE201 Operating Systems    D01  P1003            3              1
EEE101 Circuit Theory       D02  P1002            4              1
CIV101 Structural Analysis  D04  P1004            3              0
```

4.3. SQL Functions

1. Use numeric functions like ROUND, MOD, POWER on dummy values.

CODE:

```
SELECT
  ROUND(123.456, 2) AS Rounded_Value,
  MOD(10, 3) AS Modulus,
  POWER(2, 5) AS Power_Value
FROM DUAL;
```

OUTPUT:

```
SQL> SELECT
  2    ROUND(123.456, 2) AS Rounded_Value,
  3    MOD(10, 3) AS Modulus,
  4    POWER(2, 5) AS Power_Value
  5  FROM DUAL;

ROUNDED_VALUE    MODULUS POWER_VALUE
------------- ---------- -----------
       123.46          1          32
```

2. Use string functions like LENGTH, SUBSTR, INSTR, UPPER, LOWER on names in Professors and Students.

CODE:

For Professors:

```
SELECT
  Prof_Name,
  LENGTH(Prof_Name) AS Length,
  SUBSTR(Prof_Name, 1, 5) AS Substr,
  INSTR(Prof_Name, 'a') AS Pos,
  UPPER(Prof_Name) AS Uppercase,
  LOWER(Prof_Name) AS Lowercase
FROM Professors;
```

For Students:

```
SELECT
  Student_Name,
  LENGTH(Student_Name) AS Name_Length,
  SUBSTR(Student_Name, 1, 5) AS Substr,
  INSTR(Student_Name, 'a') AS Pos,
  UPPER(Student_Name) AS Uppercase,
  LOWER(Student_Name) AS Lowercase
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Prof_Name,
  3    LENGTH(Prof_Name) AS Length,
  4    SUBSTR(Prof_Name, 1, 5) AS Substr,
  5    INSTR(Prof_Name, 'a') AS Pos,
  6    UPPER(Prof_Name) AS Uppercase,
  7    LOWER(Prof_Name) AS Lowercase
  8  FROM Professors;

PROF_NAME              LENGTH SUBSTR        POS UPPERCASE        LOWERCASE
------------------ ---------- ------ ---------- ---------------- ----------------
Dr. Meera Nair             14 Dr. M           9 DR. MEERA NAIR   dr. meera nair
Dr. Arjun Rao              13 Dr. A          12 DR. ARJUN RAO    dr. arjun rao
Dr. Kavita Singh           16 Dr. K           6 DR. KAVITA SINGH dr. kavita singh
Dr. Raj Malhotra           16 Dr. R           6 DR. RAJ MALHOTRA dr. raj malhotra

SQL> SELECT
  2    Student_Name,
  3    LENGTH(Student_Name) AS Name_Length,
  4    SUBSTR(Student_Name, 1, 5) AS Substr,
  5    INSTR(Student_Name, 'a') AS Pos,
  6    UPPER(Student_Name) AS Uppercase,
  7    LOWER(Student_Name) AS Lowercase
  8  FROM Students;

STUDENT_NAME         NAME_LENGTH SUBSTR        POS UPPERCASE        LOWERCASE
-------------------- ----------- ------ ---------- ---------------- ----------------
Anjali Sharma                13 Anjal           4 ANJALI SHARMA    anjali sharma
Ravi Kumar                   10 Ravi            2 RAVI KUMAR       ravi kumar
Nisha Verma                  11 Nisha           5 NISHA VERMA      nisha verma
Aman Sheikh                  11 Aman            3 AMAN SHEIKH      aman sheikh
```

3. Use conversion functions on DOB.

CODE:

```
SELECT
  Student_ID,
  Student_Name,
  DOB,
  TO_CHAR(DOB, 'DD-MM-YYYY') AS Formatted_DOB,
  TO_CHAR(DOB, 'Month') AS Month_Name,
  TO_NUMBER(TO_CHAR(DOB, 'YYYY')) AS Year_Only
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Student_ID,
  3    Student_Name,
  4    DOB,
  5    TO_CHAR(DOB, 'DD-MM-YYYY') AS Formatted_DOB,
  6    TO_CHAR(DOB, 'Month') AS Month_Name,
  7    TO_NUMBER(TO_CHAR(DOB, 'YYYY')) AS Year_Only
  8  FROM Students;

STUDEN STUDENT_NAME         DOB       FORMATTED_DOB MONTH_NAME    YEAR_ONLY
------ -------------------- --------- ------------- ------------ ----------
S0001  Anjali Sharma        14-MAY-03 14-05-2003    May                2003
S0002  Ravi Kumar           20-NOV-02 20-11-2002    November           2002
S0003  Nisha Verma          02-FEB-03 02-02-2003    February           2003
S0004  Aman Sheikh          25-JUL-02 25-07-2002    July               2002
```

4. Count total number of students.

CODE:

```
SELECT COUNT(*) AS Total_Students FROM Students;
```

OUTPUT:

```
SQL> SELECT COUNT(*) AS Total_Students FROM Students;

TOTAL_STUDENTS
--------------
             4
```

5. Find max and min marks in Enrollments as max_marks, min_marks.

CODE:

```
SELECT
  MAX(Marks) AS Max_Marks,
  MIN(Marks) AS Min_Marks
FROM Enrollments;
```

OUTPUT:

```
SQL> SELECT
  2    MAX(Marks) AS Max_Marks,
  3    MIN(Marks) AS Min_Marks
  4  FROM Enrollments;

 MAX_MARKS  MIN_MARKS
---------- ----------
        93       68.5
```

6. Count number of students with marks over 75.

CODE:

```
SELECT COUNT(*) AS Students_Above_75
FROM Enrollments
WHERE Marks > 75;
```

OUTPUT:

```
SQL> SELECT COUNT(*) AS Students_Above_75
  2  FROM Enrollments
  3  WHERE Marks > 75;

STUDENTS_ABOVE_75
-----------------
                4
```

4.4. Date Functions

1. List student names and their day of birth.

CODE:

```
SELECT
  Student_Name,
  TO_CHAR(DOB, 'Day') AS Day_Of_Birth
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    TO_CHAR(DOB, 'Day') AS Day_Of_Birth
  4  FROM Students;

STUDENT_NAME         DAY_OF_BIRTH
-------------------- ---------------
Anjali Sharma        Wednesday
Ravi Kumar           Wednesday
Nisha Verma          Sunday
Aman Sheikh          Thursday
```

2. Format DOBs in 'DD-Month-YYYY' format.

CODE:

```
SELECT
  Student_Name,
  TO_CHAR(DOB, 'DD-Month-YYYY')
  AS Formatted_DOB
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    TO_CHAR(DOB, 'DD-Month-YYYY') AS Formatted_DOB
  4  FROM Students;

STUDENT_NAME         FORMATTED_DOB
-------------------- --------------------
Anjali Sharma        14-May      -2003
Ravi Kumar           20-November -2002
Nisha Verma          02-February -2003
Aman Sheikh          25-July     -2002
```

3. Show DOBs in 'DD-MM-YY' format.

CODE:

```
SELECT
  Student_Name,
  TO_CHAR(DOB, 'DD-MM-YY')
  AS Short_DOB
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    TO_CHAR(DOB, 'DD-MM-YY') AS Short_DOB
  4  FROM Students;

STUDENT_NAME         SHORT_DOB
-------------------- ----------
Anjali Sharma        14-05-03
Ravi Kumar           20-11-02
Nisha Verma          02-02-03
Aman Sheikh          25-07-02
```

4. Add 100 days to all DOBs.

CODE:

```
SELECT
  Student_Name,
  DOB,
  DOB + 100 AS NEW_DOB
FROM Students;
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    DOB,
  4    DOB + 100 AS NEW_DOB
  5  FROM Students;

STUDENT_NAME         DOB       NEW_DOB
-------------------- --------- ---------
Anjali Sharma        14-MAY-03 22-AUG-03
Ravi Kumar           20-NOV-02 28-FEB-03
Nisha Verma          02-FEB-03 13-MAY-03
Aman Sheikh          25-JUL-02 02-NOV-02
```

5. List students born in May.

CODE:

```
SELECT
  Student_Name,
  DOB
FROM Students
WHERE TO_CHAR(DOB, 'MM') = '05';
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    DOB
  4  FROM Students
  5  WHERE TO_CHAR(DOB, 'MM') = '05';

STUDENT_NAME         DOB
-------------------- ---------
Anjali Sharma        14-MAY-03
```

6. List students born between 2002 and 2003.

CODE:

```
SELECT
  Student_Name,
  DOB
FROM Students
WHERE DOB BETWEEN TO_DATE('2002-01-01', 'YYYY-MM-DD')
AND TO_DATE('2003-12-31', 'YYYY-MM-DD');
```

OUTPUT:

```
SQL> SELECT
  2    Student_Name,
  3    DOB
  4  FROM Students
  5  WHERE DOB BETWEEN TO_DATE('2002-01-01', 'YYYY-MM-DD')
  6  AND TO_DATE('2003-12-31', 'YYYY-MM-DD');

STUDENT_NAME         DOB
-------------------- ---------
Anjali Sharma        14-MAY-03
Ravi Kumar           20-NOV-02
Nisha Verma          02-FEB-03
Aman Sheikh          25-JUL-02
```

4.5. Set Operators

TABLE CREATION CODE:

```
CREATE TABLE Top_Courses (
  Course_Name VARCHAR2(100),
  Dept_Name VARCHAR2(50)
);
```

OUTPUT:

```
SQL> CREATE TABLE Top_Courses (
  2    Course_Name VARCHAR2(100),
  3    Dept_Name VARCHAR2(50)
  4  );

Table created.
```

SAMPLE INSERTION CODE:

```
INSERT INTO Top_Courses VALUES ('Data Structures', 'Computer Science');
INSERT INTO Top_Courses VALUES ('Circuit Theory', 'Electrical Engg.');
INSERT INTO Top_Courses VALUES ('Artificial Intelligence', 'Computer Science');
INSERT INTO Top_Courses VALUES ('Thermodynamics', 'Mechanical Engg.');
```

OUTPUT:

```
SQL> INSERT INTO Top_Courses VALUES ('Data Structures', 'Computer Science');

1 row created.

SQL> INSERT INTO Top_Courses VALUES ('Circuit Theory', 'Electrical Engg.');

1 row created.

SQL> INSERT INTO Top_Courses VALUES ('Artificial Intelligence', 'Computer Science');

1 row created.

SQL> INSERT INTO Top_Courses VALUES ('Thermodynamics', 'Mechanical Engg.');

1 row created.

SQL> SELECT * FROM Top_Courses;

COURSE_NAME               DEPT_NAME
------------------------- ------------------------
Data Structures           Computer Science
Circuit Theory            Electrical Engg.
Artificial Intelligence   Computer Science
Thermodynamics            Mechanical Engg.
```

1. Show unique course names from both Courses and Top_Courses.

CODE:

```
SELECT Course_Name FROM Courses
UNION
SELECT Course_Name FROM Top_Courses;
```

OUTPUT:

```
SQL> SELECT Course_Name FROM Courses
  2  UNION
  3  SELECT Course_Name FROM Top_Courses;

COURSE_NAME
------------------------
Data Structures
Operating Systems
Circuit Theory
Thermodynamics
Structural Analysis
Artificial Intelligence

6 rows selected.
```

2. Show common courses between Courses and Top_Courses.

CODE:

```
SELECT Course_Name FROM Courses
INTERSECT
SELECT Course_Name FROM Top_Courses;
```

OUTPUT:

```
SQL> SELECT Course_Name FROM Courses
  2  INTERSECT
  3  SELECT Course_Name FROM Top_Courses;

COURSE_NAME
------------------------
Data Structures
Circuit Theory
Thermodynamics
```

3. Show top courses that are not in current courses.

CODE:

```
SELECT Course_Name FROM Top_Courses
MINUS
SELECT Course_Name FROM Courses;
```

OUTPUT:

```
SQL> SELECT Course_Name FROM Top_Courses
  2  MINUS
  3  SELECT Course_Name FROM Courses;

COURSE_NAME
-------------------------
Artificial Intelligence
```

4. Show union all of both.

CODE:

```
SELECT Course_Name FROM Courses
UNION ALL
SELECT Course_Name FROM Top_Courses;
```

OUTPUT:

```
SQL> SELECT Course_Name FROM Courses
  2  UNION ALL
  3  SELECT Course_Name FROM Top_Courses;

COURSE_NAME
-------------------------
Data Structures
Operating Systems
Circuit Theory
Thermodynamics
Structural Analysis
Data Structures
Circuit Theory
Artificial Intelligence
Thermodynamics

9 rows selected.
```

4.6 Sorting

1. Sort professors by descending experience.

CODE:

```
SELECT *
FROM PROFESSORS
ORDER BY EXPERIENCE_YEARS DESC;
```

OUTPUT:

```
SQL> SELECT *
  2  FROM PROFESSORS
  3  ORDER BY EXPERIENCE_YEARS DESC;

PROF_ID  PROF_NAME            DEPT_ID EXPERIENCE_YEARS
-------- -------------------- ------- ----------------
P1004    Dr. Raj Malhotra     D03                  15
P1001    Dr. Meera Nair       D01                  12
P1002    Dr. Arjun Rao        D02                   9
P1003    Dr. Kavita Singh     D01                   7
```

4.7 Group By, Having

1. Number of students per department (only those >1).

CODE:

```
SELECT d.Dept_Name,
COUNT(s.Student_ID) AS Student_Count
FROM Departments d, Students s
WHERE d.Dept_ID = s.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name
HAVING COUNT(s.Student_ID) > 1;
```

OUTPUT:

```
SQL> SELECT d.Dept_Name,
  2  COUNT(s.Student_ID) AS Student_Count
  3  FROM Departments d, Students s
  4  WHERE d.Dept_ID = s.Dept_ID
  5  GROUP BY d.Dept_ID, d.Dept_Name
  6  HAVING COUNT(s.Student_ID) > 1;

DEPT_NAME          STUDENT_COUNT
------------------ -------------
Computer Science               2
```

2. Departments with avg classroom count >5.

CODE:

```
SELECT DEPT_NAME, AVG(NUMBER_OF_CLASSROOMS) AS
AVG_CLASSROOMS
FROM DEPARTMENTS
GROUP BY DEPT_NAME
HAVING AVG(NUMBER_OF_CLASSROOMS) > 5;
```

OUTPUT:

```
SQL> SELECT DEPT_NAME, AVG(NUMBER_OF_CLASSROOMS) AS AVG_CLASSROOMS
  2  FROM DEPARTMENTS
  3  GROUP BY DEPT_NAME
  4  HAVING AVG(NUMBER_OF_CLASSROOMS) > 5;

DEPT_NAME          AVG_CLASSROOMS
------------------ --------------
Computer Science               10
Electrical Engg.                8
Mechanical Engg.                6
```

3. Courses taught by professors with more than 1 course

CODE:

```
SELECT c.COURSE_NAME, p.PROF_NAME
FROM COURSES c, PROFESSORS p
WHERE c.PROF_ID = p.PROF_ID
AND c.PROF_ID IN (
   SELECT PROF_ID FROM COURSES
   GROUP BY PROF_ID HAVING COUNT(*) > 1
);
```

OUTPUT:

```
SQL> SELECT c.COURSE_NAME, p.PROF_NAME
  2  FROM COURSES c, PROFESSORS p
  3  WHERE c.PROF_ID = p.PROF_ID
  4  AND c.PROF_ID IN (
  5      SELECT PROF_ID
  6      FROM COURSES
  7      GROUP BY PROF_ID
  8      HAVING COUNT(*) > 1
  9  );

COURSE_NAME          PROF_NAME
-------------------- --------------------
Structural Analysis  Dr. Raj Malhotra
Thermodynamics       Dr. Raj Malhotra
```

4.8 Subqueries

1. Find names of students enrolled in any course.

CODE:

```
SELECT Student_Name
FROM Students
WHERE Student_ID IN (SELECT Student_ID FROM Enrollments);
```

OUTPUT:

```
SQL> SELECT Student_Name
  2  FROM Students
  3  WHERE Student_ID IN (SELECT Student_ID FROM Enrollments);

STUDENT_NAME
---------------
Anjali Sharma
Ravi Kumar
Nisha Verma
Aman Sheikh
```

2. Find department of student S0001.

CODE:

```
SELECT d.Dept_Name
FROM Departments d WHERE d.Dept_ID = (
SELECT Dept_ID  FROM Students  WHERE Student_ID = 'S0001'
);
```

OUTPUT:

```
SQL> SELECT d.Dept_Name
  2  FROM Departments d WHERE d.Dept_ID = (
  3  SELECT Dept_ID  FROM Students  WHERE Student_ID = 'S0001'
  4  );

DEPT_NAME
--------------------
Computer Science
```

3. Departments where student count > 1.

CODE:

```
SELECT d.Dept_Name
FROM Departments d
WHERE d.Dept_ID IN (
  SELECT Dept_ID FROM Students
  GROUP BY Dept_ID HAVING COUNT(*) > 1
);
```

OUTPUT:

```
SQL> SELECT d.Dept_Name
  2  FROM Departments d
  3  WHERE d.Dept_ID IN (
  4     SELECT Dept_ID FROM Students GROUP BY Dept_ID HAVING COUNT(*) > 1
  5  );

DEPT_NAME
------------------
Computer Science
```

4. List courses taught by professors with > 10 years experience.

CODE:

```
SELECT Course_Name
FROM Courses
WHERE Prof_ID IN (
SELECT Prof_ID FROM Professors WHERE Experience_Years > 10);
```

OUTPUT:

```
SQL> SELECT Course_Name
  2  FROM Courses
  3  WHERE Prof_ID IN (SELECT Prof_ID FROM Professors WHERE Experience_Years > 10);

COURSE_NAME
--------------------
Data Structures
Structural Analysis
Thermodynamics
```

5. List students in departments with at least 1 professor.

CODE:

```
SELECT Student_Name
FROM Students
WHERE Dept_ID IN (SELECT DISTINCT Dept_ID FROM Professors);
```

OUTPUT:

```
SQL> SELECT Student_Name
  2  FROM Students
  3  WHERE Dept_ID IN (SELECT DISTINCT Dept_ID FROM Professors);

STUDENT_NAME
--------------------
Anjali Sharma
Aman Sheikh
Ravi Kumar
Nisha Verma
```

6. Find the professors with exp less than the average experience of all the professors.

CODE:

```
SELECT Prof_Name
FROM Professors
WHERE Experience_Years < ( SELECT AVG(Experience_Years)
FROM Professors);
```

OUTPUT:

```
SQL> SELECT Prof_Name
  2  FROM Professors
  3  WHERE Experience_Years < ( SELECT AVG(Experience_Years)
  4  FROM Professors);

PROF_NAME
--------------------
Dr. Arjun Rao
Dr. Kavita Singh
```

4.9 NOT EXISTS

1. Departments where every professor has > 5 years experience.

CODE:

```
SELECT d.Dept_Name FROM Departments d
WHERE NOT EXISTS (
   SELECT 1 FROM Professors p
   WHERE p.Dept_ID = d.Dept_ID AND p.Experience_Years <= 5 );
```

OUTPUT:

```
SQL> SELECT d.Dept_Name FROM Departments d
  2  WHERE NOT EXISTS (
  3      SELECT 1 FROM Professors p
  4      WHERE p.Dept_ID = d.Dept_ID AND p.Experience_Years <= 5 );

DEPT_NAME
------------------
Computer Science
Civil Engg
Electrical Engg.
Mechanical Engg.
```

2. Departments with no students.

CODE:

```
SELECT d.Dept_Name
FROM Departments d
WHERE NOT EXISTS ( SELECT 1
FROM Students s WHERE s.Dept_ID = d.Dept_ID );
```

OUTPUT:

```
SQL> SELECT d.Dept_Name
  2  FROM Departments d
  3  WHERE NOT EXISTS ( SELECT 1
  4  FROM Students s WHERE s.Dept_ID = d.Dept_ID );

DEPT_NAME
------------------
Civil Engg
```

4.10 Correlated Subqueries

1. Students with marks above average.

CODE:

```
SELECT DISTINCT s.Student_Name
FROM Students s
JOIN Enrollments e ON s.Student_ID = e.Student_ID
WHERE e.Marks > (SELECT AVG(Marks) FROM Enrollments);
```

OUTPUT:

```
SQL> SELECT DISTINCT s.Student_Name
  2  FROM Students s
  3  JOIN Enrollments e ON s.Student_ID = e.Student_ID
  4  WHERE e.Marks > (SELECT AVG(Marks) FROM Enrollments);

STUDENT_NAME
------------------
Anjali Sharma
Nisha Verma
```

2. Students with DOB later than any student from 'Electrical Engg'.

CODE:

```
SELECT s.Student_Name
FROM Students s
WHERE s.DOB > ANY (
   SELECT s2.DOB
   FROM Students s2
   JOIN Departments d ON s2.Dept_ID = d.Dept_ID
   WHERE d.Dept_Name = 'Electrical Engg.'
);
```

OUTPUT:

```
SQL> SELECT s.Student_Name
  2  FROM Students s
  3  WHERE s.DOB > ANY (
  4      SELECT s2.DOB
  5      FROM Students s2
  6      JOIN Departments d ON s2.Dept_ID = d.Dept_ID
  7      WHERE d.Dept_Name = 'Electrical Engg.'
  8  );

STUDENT_NAME
------------------
Anjali Sharma
Nisha Verma
```

4.11 JOINING Tables

1. List student names along with their department name.

CODE:

```
SELECT s.Student_Name, d.Dept_Name
FROM Students s
INNER JOIN Departments d
   ON s.Dept_ID = d.Dept_ID;
```

OUTPUT:

```
SQL> SELECT s.Student_Name, d.Dept_Name
  2  FROM Students s
  3  INNER JOIN Departments d
  4       ON s.Dept_ID = d.Dept_ID;

STUDENT_NAME       DEPT_NAME
------------------ ------------------
Anjali Sharma      Computer Science
Aman Sheikh        Computer Science
Ravi Kumar         Electrical Engg.
Nisha Verma        Mechanical Engg.
```

2. Courses and number of students enrolled.

CODE:

```
SELECT c.Course_Name, COUNT(e.Student_ID) AS Student_Count
FROM Courses c
LEFT JOIN Enrollments e ON c.Course_ID = e.Course_ID
GROUP BY c.Course_Name;
```

OUTPUT:

```
SQL> SELECT c.Course_Name, COUNT(e.Student_ID) AS Student_Count
  2  FROM Courses c
  3  LEFT JOIN Enrollments e ON c.Course_ID = e.Course_ID
  4  GROUP BY c.Course_Name;

COURSE_NAME          STUDENT_COUNT
-------------------- -------------
Structural Analysis              0
Data Structures                  2
Operating Systems                1
Circuit Theory                   1
Thermodynamics                   1
```

3. List course name and professor name.

CODE:

```
SELECT c.Course_Name, p.Prof_Name
FROM Courses c
INNER JOIN Professors p
    ON c.Prof_ID = p.Prof_ID;
```

OUTPUT:

```
SQL> SELECT c.Course_Name, p.Prof_Name
  2  FROM Courses c
  3  INNER JOIN Professors p
  4          ON c.Prof_ID = p.Prof_ID;

COURSE_NAME          PROF_NAME
-------------------- --------------------
Data Structures      Dr. Meera Nair
Circuit Theory       Dr. Arjun Rao
Operating Systems    Dr. Kavita Singh
Structural Analysis  Dr. Raj Malhotra
Thermodynamics       Dr. Raj Malhotra
```

4. Departments with more than one professor.

CODE:

```
SELECT d.Dept_Name FROM Departments d
INNER JOIN Professors p ON d.Dept_ID = p.Dept_ID
GROUP BY d.Dept_Name HAVING COUNT(p.Prof_ID) > 1;
```

OUTPUT:

```
SQL> SELECT d.Dept_Name FROM Departments d
  2  INNER JOIN Professors p ON d.Dept_ID = p.Dept_ID
  3  GROUP BY d.Dept_Name HAVING COUNT(p.Prof_ID) > 1;

DEPT_NAME
--------------------
Computer Science
```

5. List students enrolled in courses with more than 3 credits.

CODE:

```
SELECT DISTINCT s.Student_Name
FROM Students s
INNER JOIN Enrollments e ON s.Student_ID = e.Student_ID
INNER JOIN Courses c ON e.Course_ID = c.Course_ID
WHERE c.Credits > 3;
```

OUTPUT:

```
SQL> SELECT DISTINCT s.Student_Name
  2  FROM Students s
  3  INNER JOIN Enrollments e ON s.Student_ID = e.Student_ID
  4  INNER JOIN Courses c ON e.Course_ID = c.Course_ID
  5  WHERE c.Credits > 3;

STUDENT_NAME
-----------------
Anjali Sharma
Aman Sheikh
Ravi Kumar
```

5. PL/SQL Section

5.1 Basic Blocks

1. Accept a course ID and show number of students enrolled.

CODE:

```
SET SERVEROUTPUT ON;
SET VERIFY OFF;

DECLARE
   v_course_id  VARCHAR2(6) := '&c_id';
   v_count      NUMBER;

BEGIN
   SELECT COUNT(*) INTO v_count
   FROM Enrollments
   WHERE Course_ID = v_course_id;

   DBMS_OUTPUT.PUT_LINE('Course ' || v_course_id || ' has ' || v_count || ' students enrolled.');
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL1.sql"
Enter value for c_id: CSE101
Course CSE101 has 2 students enrolled.

PL/SQL procedure successfully completed.
```

2. If department 'Biotech' doesn't exist, insert it with ‘D05’.

CODE:

```
SET SERVEROUTPUT ON;


DECLARE
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM Departments
   WHERE Dept_Name = 'Biotech';


   IF v_count = 0 THEN
      INSERT INTO Departments
      VALUES ('D05', 'Biotech', 'Bio Block', 5);
      DBMS_OUTPUT.PUT_LINE('Department Biotech inserted with ID D05.');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Department Biotech already exists.');
   END IF;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL2.sql"
Department Biotech inserted with ID D05.

PL/SQL procedure successfully completed.

SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL2.sql"
Department Biotech already exists.

PL/SQL procedure successfully completed.
```

3. Calculate total students in ‘Computer Science’.

CODE:

```
SET SERVEROUTPUT ON;


DECLARE
   v_total NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_total
   FROM Students s
   JOIN Departments d ON s.Dept_ID = d.Dept_ID
   WHERE d.Dept_Name = 'Computer Science';


   DBMS_OUTPUT.PUT_LINE('Total students in Computer Science: ' || v_total);
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL3.sql"
Total students in Computer Science: 2

PL/SQL procedure successfully completed.
```

5.2 Cursors

1. Insert a new student and show row count.

CODE:

```
SET SERVEROUTPUT ON;


DECLARE
   v_rows NUMBER;
BEGIN
   INSERT INTO Students
   VALUES ('S0005', 'Rohit Sharma', 'D01', TO_DATE('2003-08-15', 'YYYY-MM-DD'));


   v_rows := SQL%ROWCOUNT;
   DBMS_OUTPUT.PUT_LINE('Number of rows inserted: ' || v_rows);


   COMMIT;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL4.sql"
Number of rows inserted: 1

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Students;

STUDEN STUDENT_NAME                                        DEPT DOB
------ --------------------------------------------------- ---- ---------
S0005  Rohit Sharma                                        D01  15-AUG-03
S0001  Anjali Sharma                                       D01  14-MAY-03
S0002  Ravi Kumar                                          D02  20-NOV-02
S0003  Nisha Verma                                         D03  02-FEB-03
S0004  Aman Sheikh                                         D01  25-JUL-02
```

2. Update marks and show affected row count.

CODE:

```
SET SERVEROUTPUT ON;


DECLARE
   v_rows NUMBER;
BEGIN
   UPDATE Enrollments
   SET Marks = 85
   WHERE Student_ID = 'S0004' AND Course_ID = 'CSE101';


   v_rows := SQL%ROWCOUNT;
   DBMS_OUTPUT.PUT_LINE('Number of rows updated: ' || v_rows);


   COMMIT;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL5.sql"
Number of rows updated: 1

PL/SQL procedure successfully completed.
```

3. List students with marks above 80 (Explicit Cursor).

CODE:

```
SET SERVEROUTPUT ON;

DECLARE
   CURSOR c_high_marks IS
      SELECT s.Student_ID, s.Student_Name, e.Course_ID, e.Marks
      FROM Students s JOIN Enrollments e ON s.Student_ID = e.Student_ID
      WHERE e.Marks > 80;
   v_student c_high_marks%ROWTYPE;
BEGIN
   OPEN c_high_marks;
   LOOP
      FETCH c_high_marks INTO v_student;
      EXIT WHEN c_high_marks%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Student: ' || v_student.Student_Name ||
                           ', Course: ' || v_student.Course_ID ||
                           ', Marks: ' || v_student.Marks);
   END LOOP;
   CLOSE c_high_marks;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL6.sql"
Student: Anjali Sharma, Course: CSE101, Marks: 88
Student: Ravi Kumar, Course: EEE101, Marks: 81
Student: Nisha Verma, Course: ME101, Marks: 93
Student: Aman Sheikh, Course: CSE101, Marks: 85

PL/SQL procedure successfully completed.
```

4. List all courses taught by P1001 (Explicit Cursor).

CODE:

```
SET SERVEROUTPUT ON;


DECLARE
   CURSOR c_courses IS
      SELECT Course_ID, Course_Name
      FROM Courses
      WHERE Prof_ID = 'P1001';


   v_course c_courses%ROWTYPE;
BEGIN
   OPEN c_courses;
   LOOP
      FETCH c_courses INTO v_course;
      EXIT WHEN c_courses%NOTFOUND;


      DBMS_OUTPUT.PUT_LINE('Course ID: ' || v_course.Course_ID ||
                           ', Name: ' || v_course.Course_Name);
   END LOOP;
   CLOSE c_courses;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL7.sql"
Course ID: CSE101, Name: Data Structures

PL/SQL procedure successfully completed.
```

5.3 Procedures

1. Procedure to accept student ID and return number of enrolled courses.

CODE:

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE get_enrolled_courses IS
   v_student_id Students.Student_ID%TYPE;
   v_count NUMBER;
BEGIN
   v_student_id := '&stud_id';


   SELECT COUNT(*) INTO v_count
   FROM Enrollments WHERE Student_ID = v_student_id;


   DBMS_OUTPUT.PUT_LINE('Student ' || v_student_id ||
                        ' is enrolled in ' || v_count || ' courses.');
END;
/

BEGIN
   get_enrolled_courses;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL8.sql"
Enter value for stud_id: S0001

Procedure created.

Student S0001 is enrolled in 2 courses.

PL/SQL procedure successfully completed.
```

2. Procedure to insert new course if department and professor exist.

CODE:

```
SET SERVEROUTPUT ON;


CREATE OR REPLACE PROCEDURE insert_course_if_valid (
   p_course_id IN Courses.Course_ID%TYPE,
   p_course_name IN Courses.Course_Name%TYPE,
   p_dept_id IN Courses.Dept_ID%TYPE,
   p_prof_id IN Courses.Prof_ID%TYPE,
   p_credits IN NUMBER,
   p_no_of_students IN NUMBER
) IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count FROM Departments WHERE Dept_ID =
p_dept_id;
   IF v_count = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Department does not exist!');
      RETURN;
   END IF;


   SELECT COUNT(*) INTO v_count FROM Professors WHERE Prof_ID =
p_prof_id;
   IF v_count = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Professor does not exist!');
      RETURN;
   END IF;


   INSERT INTO Courses
```

```
    VALUES (p_course_id, p_course_name, p_dept_id, p_prof_id, p_credits,
p_no_of_students);
    DBMS_OUTPUT.PUT_LINE('Course inserted successfully!');
END;
/


BEGIN
    insert_course_if_valid('CSE401', 'Database Systems', 'D01', 'P1001', 4, 2);
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL9.sql"

Procedure created.

Course inserted successfully!

PL/SQL procedure successfully completed.

SQL> select * from courses;

COURSE COURSE_NAME          DEPT PROF_    CREDITS TOTAL_STUDENTS
------ -------------------- ---- ----- ---------- --------------
CSE401 Database Systems     D01  P1001          4              2
CSE101 Data Structures      D01  P1001          4              2
CSE201 Operating Systems    D01  P1003          3              1
EEE101 Circuit Theory       D02  P1002          5              1
ME101  Thermodynamics       D03  P1004          3              1
CIV101 Structural Analysis  D04  P1004          3              0

6 rows selected.
```

3. Procedure to update course credits and log to new table.

CODE:

```
CREATE TABLE Course_Log (
   Log_ID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   Course_ID VARCHAR2(10),
   Old_Credits NUMBER,
   New_Credits NUMBER,
   Updated_On DATE
);

CREATE OR REPLACE PROCEDURE update_course_credits (
   p_course_id IN Courses.Course_ID%TYPE,
   p_new_credits IN NUMBER
) IS
   v_old_credits NUMBER;
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM Courses
   WHERE Course_ID = p_course_id;

   IF v_count = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Course ID not found: ' || p_course_id);
      RETURN;
   END IF;

   SELECT Credits INTO v_old_credits
   FROM Courses
   WHERE Course_ID = p_course_id;
```

```
    UPDATE Courses
    SET Credits = p_new_credits
    WHERE Course_ID = p_course_id;

    INSERT INTO Course_Log (Course_ID, Old_Credits, New_Credits, Updated_On)
    VALUES (p_course_id, v_old_credits, p_new_credits, SYSDATE);

    DBMS_OUTPUT.PUT_LINE('Credits updated and logged successfully for course:
' || p_course_id);
END;
/

BEGIN
    update_course_credits('EEE101', 5);
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL10.sql"

PL/SQL procedure successfully completed.


Table created.


Procedure created.

Credits updated and logged successfully for course: EEE101

PL/SQL procedure successfully completed.
```

5.4 Functions

1. Function to return age of student.

CODE:

```
CREATE OR REPLACE FUNCTION get_student_age (
   p_student_id IN Students.Student_ID%TYPE
) RETURN NUMBER
IS
   v_dob DATE;
   v_age NUMBER;

BEGIN
   SELECT DOB INTO v_dob
   FROM Students
   WHERE Student_ID = p_student_id;

   v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, v_dob) / 12);

   RETURN v_age;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN NULL;
END;
/

DECLARE
   v_age NUMBER;
```

```
BEGIN
    v_age := get_student_age('S0001');
    DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL11.sql"

Function created.

Age: 22

PL/SQL procedure successfully completed.
```

2. Function to count courses offered by a department.

CODE:

```
CREATE OR REPLACE FUNCTION count_courses_by_dept (
    p_dept_id IN Courses.Dept_ID%TYPE
) RETURN NUMBER
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM Courses WHERE Dept_ID = p_dept_id;
    RETURN v_count;
END;
/
DECLARE
    v_total NUMBER;
BEGIN
    v_total := count_courses_by_dept('D01');
    DBMS_OUTPUT.PUT_LINE('Total courses: ' || v_total);
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL12.sql"

Function created.

Total courses: 3

PL/SQL procedure successfully completed.
```

3. Function to check if a professor teaches more than 2 course.

CODE:

```
CREATE OR REPLACE FUNCTION teaches_more_than_two_courses (
   p_prof_id IN Courses.Prof_ID%TYPE
) RETURN VARCHAR2
IS
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM Courses
   WHERE Prof_ID = p_prof_id;

   IF v_count >= 2 THEN
      RETURN 'Teaches';
   ELSE
      RETURN 'Does not teaches';
   END IF;
END;
/

DECLARE
   v_result VARCHAR2(20);
BEGIN
   v_result := teaches_more_than_two_courses('P1004');
   DBMS_OUTPUT.PUT_LINE(v_result || ' more than 2 courses ');
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL13.sql"

Function created.

Teaches more than 2 courses
```

5.5 Triggers

1. Prevent deletion of courses with more than 2 credits.

CODE:

```
CREATE OR REPLACE TRIGGER prevent_course_deletion
BEFORE DELETE ON Courses
FOR EACH ROW
BEGIN
   IF :OLD.Credits > 2 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Cannot delete courses with more than 2 credits.');
   END IF;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL14.sql"

Trigger created.

SQL> DELETE FROM Courses
  2  WHERE Course_ID = 'EEE101';
DELETE FROM Courses
            *
ERROR at line 1:
ORA-20001: Cannot delete courses with more than 2 credits.
ORA-06512: at "C##ASHBIII.PREVENT_COURSE_DELETION", line 3
ORA-04088: error during execution of trigger
'C##ASHBIII.PREVENT_COURSE_DELETION'
```

2. Log updates to Marks in Enrollments_Log.

CODE:

```
CREATE TABLE Enrollments_Log (
   Log_ID NUMBER PRIMARY KEY,
   Student_ID VARCHAR2(10),
   Course_ID VARCHAR2(10),
   Old_Marks NUMBER,
   New_Marks NUMBER,
   Updated_On DATE
);

CREATE SEQUENCE Enrollments_Log_seq
START WITH 1
INCREMENT BY 1;

SET SERVEROUTPUT ON;

CREATE OR REPLACE TRIGGER log_marks_update
AFTER UPDATE OF Marks
ON Enrollments
FOR EACH ROW
BEGIN
   INSERT INTO Enrollments_Log (Log_ID, Student_ID, Course_ID, Old_Marks, New_Marks, Updated_On)
   VALUES (Enrollments_Log_seq.NEXTVAL, :OLD.Student_ID, :OLD.Course_ID, :OLD.Marks, :NEW.Marks, SYSDATE);
END;
/
```

OUTPUT:

```
SQL> -- Update marks to fire trigger
SQL> UPDATE Enrollments
  2  SET Marks = 85
  3  WHERE Student_ID = 'S0001' AND Course_ID = 'CSE101';

1 row updated.

SQL> -- Check log table
SQL> SELECT * FROM Enrollments_Log
  2  WHERE Student_ID = 'S0001' AND Course_ID = 'CSE101';

    LOG_ID STUDENT_ID COURSE_ID   OLD_MARKS  NEW_MARKS UPDATED_O
---------- ---------- ---------- ---------- ---------- ---------
         1 S0001      CSE101             85         85 03-OCT-25
```

3. Prevent duplicate enrollments in a course.

CODE:

```
CREATE OR REPLACE TRIGGER prevent_duplicate_enrollment
BEFORE INSERT ON Enrollments
FOR EACH ROW
DECLARE
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count FROM Enrollments
   WHERE Student_ID = :NEW.Student_ID
     AND Course_ID  = :NEW.Course_ID;


   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Duplicate enrollment not allowed.');
   END IF;
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL16.sql"

Trigger created.

SQL> INSERT INTO Enrollments(Student_ID, Course_ID, Marks)
  2  VALUES ('S0002', 'CSE101', 80);

1 row created.

SQL> INSERT INTO Enrollments(Student_ID, Course_ID, Marks)
  2  VALUES ('S0002', 'CSE101', 90);
INSERT INTO Enrollments(Student_ID, Course_ID, Marks)
            *
ERROR at line 1:
ORA-20002: Duplicate enrollment not allowed.
ORA-06512: at "C##ASHBIII.PREVENT_DUPLICATE_ENROLLMENT", line 10
ORA-04088: error during execution of trigger
'C##ASHBIII.PREVENT_DUPLICATE_ENROLLMENT'
```

4. Prevent update / delete being done on Sunday / Saturday.

CODE:

```
CREATE OR REPLACE TRIGGER block_weekend_changes
BEFORE DELETE OR UPDATE
ON Courses
BEGIN
   IF TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('SAT','SUN') THEN
      RAISE_APPLICATION_ERROR(-20003, 'Update/Delete operations are not allowed on weekends.');
   END IF;
END;
/
```

OUTPUT:

Attempt an UPDATE on a weekday (Mon–Fri):

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL17.sql"

Trigger created.

SQL> -- Attempt an UPDATE on a weekday (Mon-Fri):
SQL> UPDATE Courses
  2  SET Credits = 5
  3  WHERE Course_ID = 'CSE101';

1 row updated.
```

Attempt a DELETE on a weekend (Sat or Sun):

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL17.sql"

Trigger created.

SQL> -- Attempt a DELETE on a weekend (Sat or Sun):
SQL> DELETE FROM Courses
  2  WHERE Course_ID = 'CSE101';
DELETE FROM Courses
            *
ERROR at line 1:
ORA-20003: Update/Delete operations are not allowed on weekends.
ORA-06512: at "C##ASHBIII.BLOCK_WEEKEND_CHANGES", line 3
ORA-04088: error during execution of trigger 'C##ASHBIII.BLOCK_WEEKEND_CHANGES'
```

5.6 Views

1. View for Students and number of courses they are enrolled in.

CODE:

```
BEGIN
   EXECUTE IMMEDIATE '
      CREATE OR REPLACE VIEW Student_Course_Count AS
      SELECT s.Student_ID,
           s.Student_Name,
           COUNT(e.Course_ID) AS Num_Courses
      FROM Students s
      LEFT JOIN Enrollments e
           ON s.Student_ID = e.Student_ID
      GROUP BY s.Student_ID, s.Student_Name
   ';
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL18.sql"

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Student_Course_Count;

STUDEN STUDENT_NAME                                        NUM_COURSES
------ -------------------------------------------------- -----------
S0001  Anjali Sharma                                               2
S0002  Ravi Kumar                                                  2
S0003  Nisha Verma                                                 1
S0004  Aman Sheikh                                                 1
S0005  Rohit Sharma                                                0
```

2. View for Courses and average marks of enrolled students (only for courses with >1 student).

CODE:

```
BEGIN
   EXECUTE IMMEDIATE '
      CREATE OR REPLACE VIEW Course_Avg_Marks AS
      SELECT c.Course_ID,
           c.Course_Name,
           ROUND(AVG(e.Marks), 2) AS Avg_Marks,
           COUNT(e.Student_ID) AS Num_Students
      FROM Courses c
      JOIN Enrollments e
         ON c.Course_ID = e.Course_ID
      GROUP BY c.Course_ID, c.Course_Name
      HAVING COUNT(e.Student_ID) > 1
   ';
END;
/
```

OUTPUT:

```
SQL> @"C:\Users\ASUS\Documents\00 MCA\DBMS\PL SQL\PL19.sql"

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Course_Avg_Marks;

COURSE COURSE_NAME          AVG_MARKS NUM_STUDENTS
------ -------------------- ---------- ------------
CSE101 Data Structures           83.33            3
```

6. Project: Event Photography Order Tracking System

Abstract

The Event Photography Order Tracking System is a database management solution developed using Microsoft Access to streamline and automate the core operations of a photography studio. It efficiently manages clients, events, packages, orders, payments, and deliveries within a single integrated platform. Traditionally, photography studios depend on manual registers or spreadsheets to record bookings, payments, and deliveries, which often results in data redundancy, confusion, and delays. This project eliminates such inefficiencies by providing a structured and user-friendly system to handle all studio-related activities seamlessly.

The system is built on six interrelated tables — Client, Event, Package, Order, Payment, and Delivery — all connected through proper relationships and referential integrity. This design ensures accuracy, consistency, and easy data retrieval. Forms created using the Access Form Wizard make data entry and navigation smooth and accessible even for non-technical users. A Main Menu Form serves as the central navigation hub, allowing users to open various forms and reports easily. Combo boxes and dropdown lists simplify user interaction by reducing repetitive input and ensuring valid data selection. Queries such as Upcoming Events, Pending Payments, Revenue Summary, and Package Popularity provide insightful reports that help monitor business performance and identify trends.

Reports generated from these queries are designed in a clear and printable format for professional use. For instance, the Upcoming Events Report displays scheduled events along with client details, while the Payment Summary Report tracks payment completion and outstanding balances. Overall, the Event Photography Order Tracking System demonstrates how Microsoft Access can be effectively utilized to create a practical, efficient, and organized database application. It reduces manual effort, prevents data duplication, and enhances decision-making by offering quick insights into the studio's operations and performance.

Screenshots

1. Tables

Figure 1.1: Client Table

Client

ClientID	FullName	Phone	Email	Address
1	Jees Thomas Cleetus	9372862937	jeees@gmail.com	Vypin
2	Kishore M	9123456789	kishore@gmail.com	Chalakudy
3	Jasin Joji	9988776655	jasin@gmail.com	Puthukkad
4	Merlin Xavier	9765432109	merlie@gmail.com	Kochi
5	Ayush	9374628262	ayush@gmail.com	Elanthikkara
(New)				

Displays all clients with columns such as *ClientID, ClientName, ContactNumber, Email,* and *Address*.

Used to store and manage basic information of every customer of the photography studio.

Figure 1.2: Event Table

Event

EventID	ClientID	EventType	EventDate	EventLocatic	Photographe	Status
1	1	Wedding	10-11-2025	Hyatt	Diya Mary	Booked
2	2	Birthday	22-11-2025	Adlux	Rena Elza	Booked
3	3	Corporate	25-10-2025	CG Auditorium	Ajay	Completed
4	4	Engagement	05-12-2025	Flora	Ashbi	Booked
5	5	Baptism	22-10-2025	Kurumassery	Rosmy	Booked
(New)						

Contains *EventID, ClientID, EventType, EventDate,* and *Location*.

Used to store information of photography events linked to each client.

Figure 1.3: Package Table

Package

PackageID	PackageName	Description	Price
1	Wedding Gold	Full day shoot + album + video highlight	₹ 1,500.00
2	Birthday Silver	3-hour shoot + 100 edited photos	₹ 700.00
3	Corporate Basic	2-hour coverage + digital delivery	₹ 500.00
4	Engagement Deluxe	5-hour coverage + album	₹ 1,000.00
(New)			

Columns include *PackageID, PackageName, Description,* and *Price*.

Holds the list of available service packages offered by the studio.

Figure 1.4: Order Table

Order

OrderID	ClientID	EventID	PackageID	OrderDate	TotalAmount	Notes
1	1	1	1	01-10-2025	₹ 1,500.00	Include drone footage
2	2	2	2	10-10-2025	₹ 700.00	Extra prints required
3	3	3	3	30-09-2025	₹ 500.00	Need express delivery
4	4	4	4	12-10-2025	₹ 1,000.00	Album to be mailed overseas
5	5	5	3	22-10-2025	₹ 10,000.00	No
6	2	1	1	22-10-2025	₹ 2,000.00	No
(New)						

Includes *OrderID, ClientID, EventID, PackageID, OrderDate,* and *TotalAmount*. Maintains records of all client orders and connects clients, events, and packages together.

Figure 1.5: Payment Table

Payment

PaymentID	OrderID	PaymentDat	AmountPaid	PaymentMet	BalanceDue
1	1	05-10-2025	₹ 1,000.00	Card	₹ 500.00
2	2	11-10-2025	₹ 700.00	UPI	₹ 0.00
3	3	01-10-2025	₹ 500.00	Cash	₹ 0.00
4	4	15-10-2025	₹ 600.00	UPI	₹ 400.00
5	5	22-10-2025	₹ 10,000.00	UPI	₹ 0.00
6	6	22-10-2025	₹ 20,000.00	Cash	₹ 0.00
(New)					

Contains *PaymentID, OrderID, PaymentDate, AmountPaid,* and *PaymentMode.* Used to record payments received for each order and monitor financial transactions.

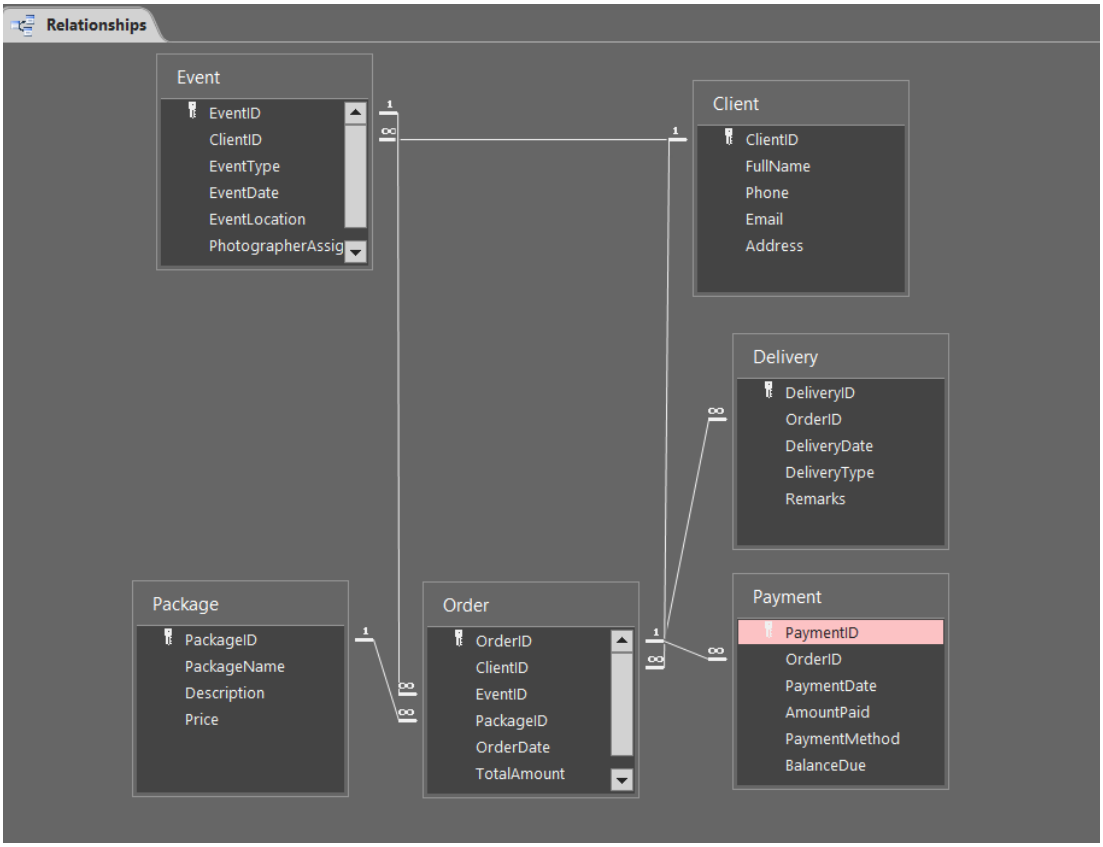
Figure 1.6: Delivery Table

Delivery

DeliveryID	OrderID	DeliveryDate	DeliveryType	Remarks
1	3	21-11-2025	Album	Client satisfied
2	2	25-11-2025	Online Link	Sent via email
3	3	05-10-2025	USB Drive	Delivered personally
4	4	20-12-2025	Album + Link	Pending delivery confirmation
5	5	22-10-2025	USB Drive	Satisfied
6	6	22-10-2025	Online Link	Satisfied.
(New)				

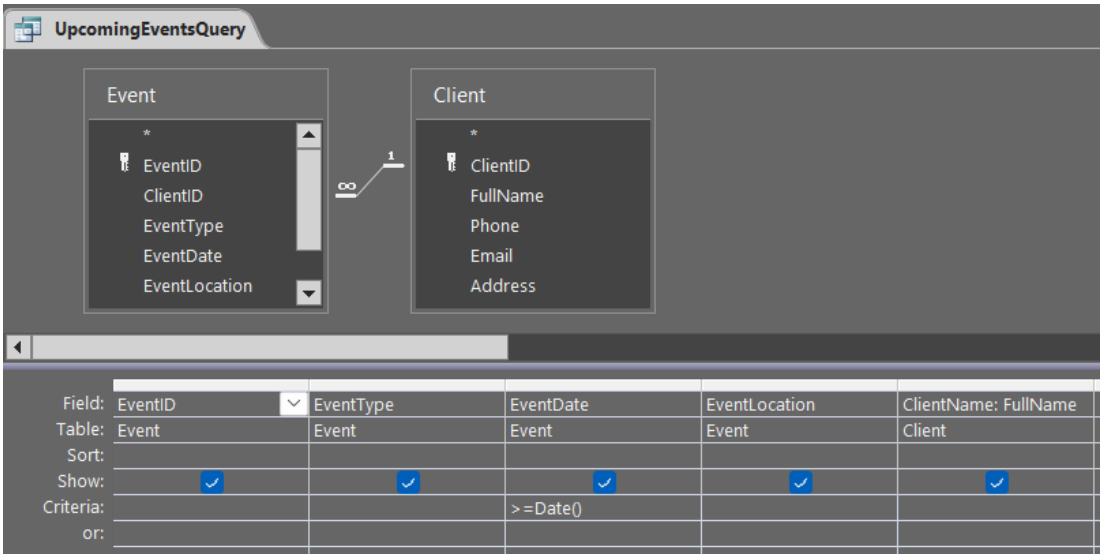
Fields include *DeliveryID, OrderID, DeliveryDate, DeliveryType,* and *Remarks.* Tracks delivery completion and type of output (USB, Online, Album) for each order.

Figure 1.8: Table Relationships



2. Queries

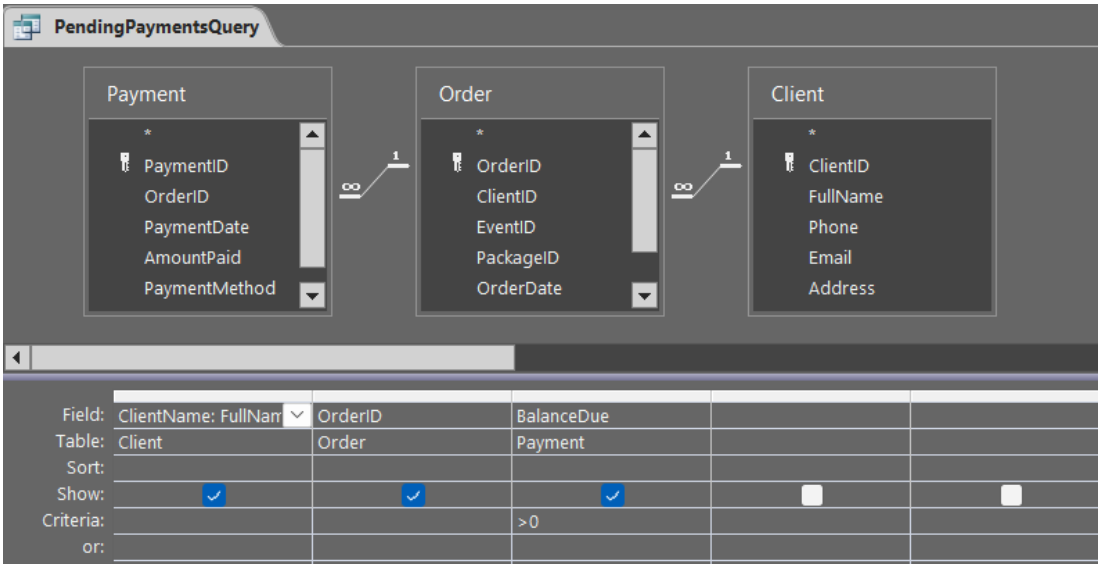
Figure 2.1: Upcoming Events Query



Displays future events by filtering *EventDate >= Date()*, showing *ClientName, EventType,* and *EventDate.*

Output: A list of all upcoming photography bookings scheduled for future dates.

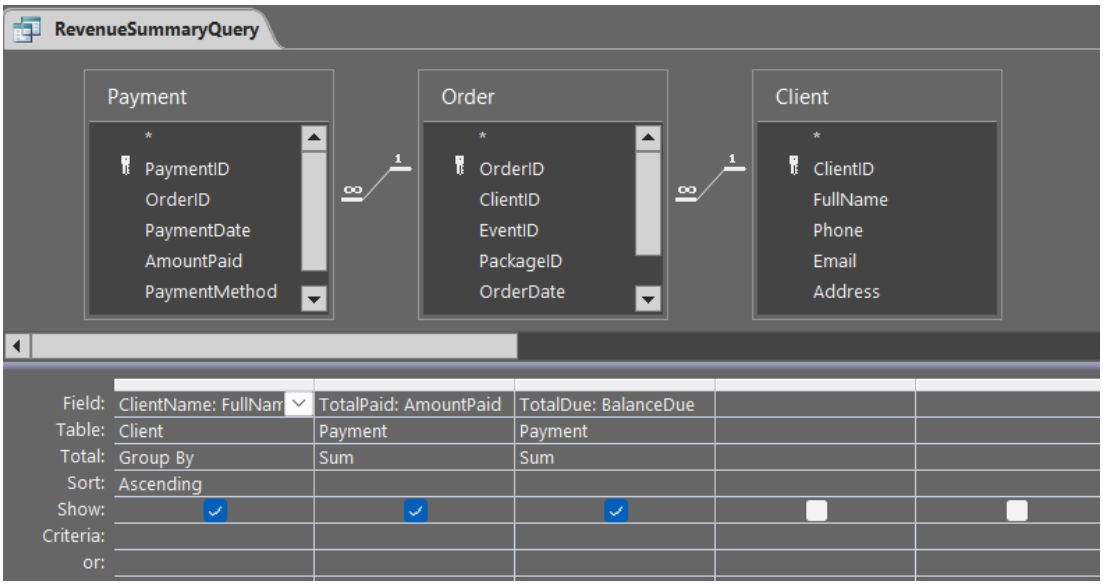
Figure 2.2: Pending Payments Query



Lists orders where the total amount exceeds the sum of payments made.

Output: Displays *ClientName, OrderID, TotalAmount, AmountPaid,* and *Balance* to find pending dues.

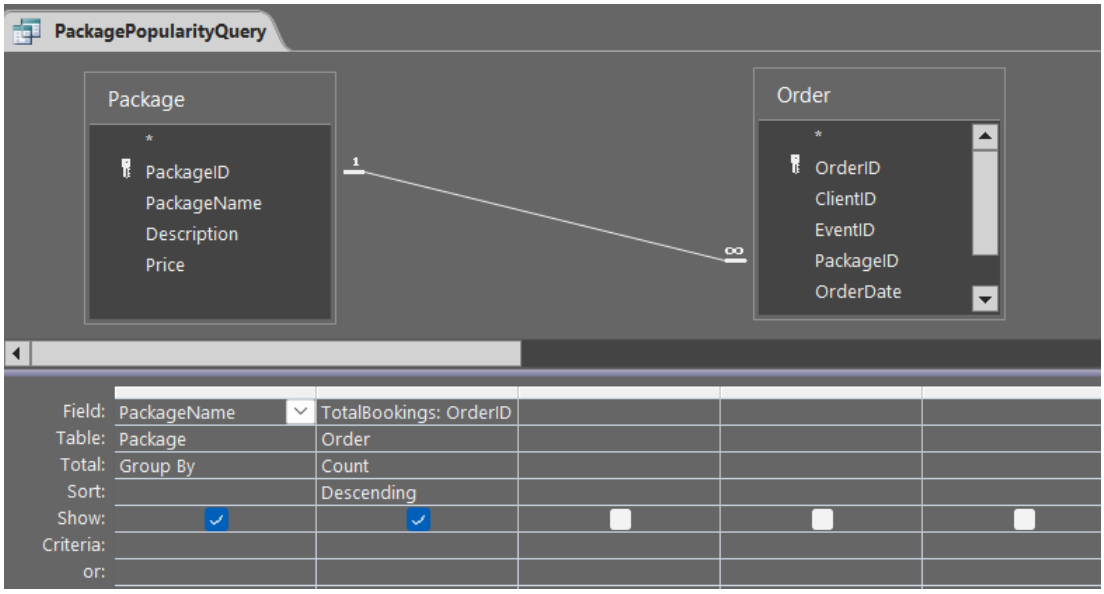
Figure 2.3: Revenue Summary Query



Summarizes *ClientName, OrderID, PackageName,* and *TotalAmount.*

Output: A clear view of total revenue earned from each client or order.

Figure 2.4: Package Popularity Query

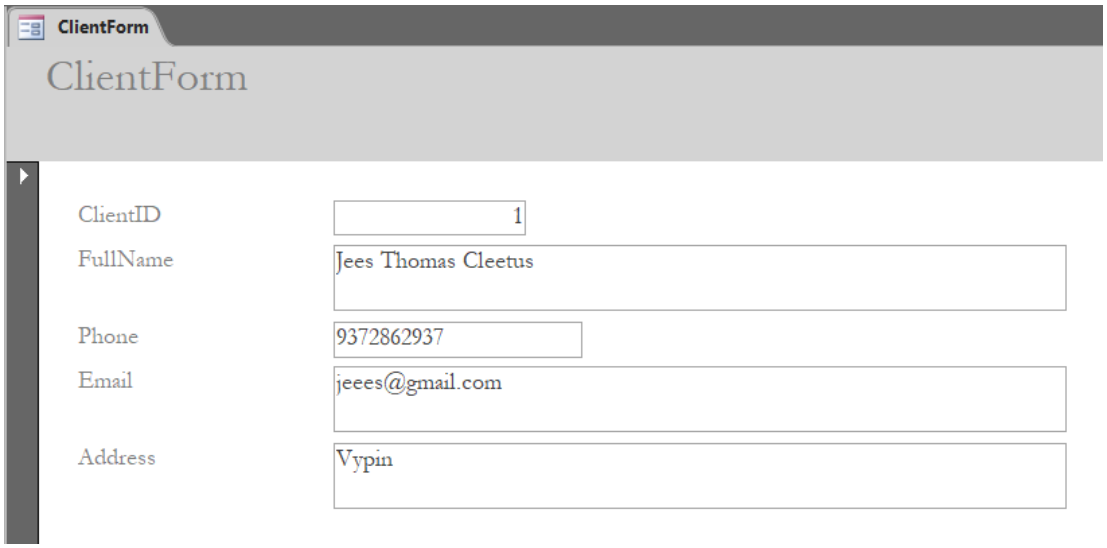


Summarizes PackageName, TimesOrdered, and TotalRevenue for each photography package.

Output: Provides a clear view of how many times each package was booked and the total income generated from it.

3. Forms

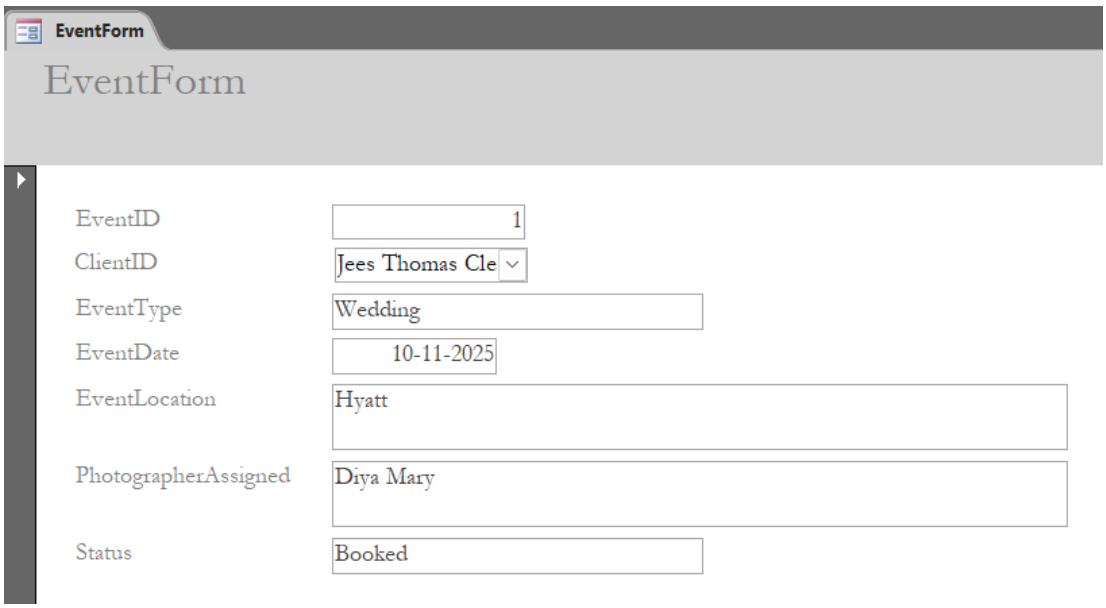
Figure 3.1: Client Form



Used to easily add, edit, or delete client details without opening the datasheet.

Simplifies client data entry and reduces chances of input errors.

Figure 3.2: Event Form



Used to enter event details like event type, date, and location for each client.

Ensures every event is properly associated with a client record.

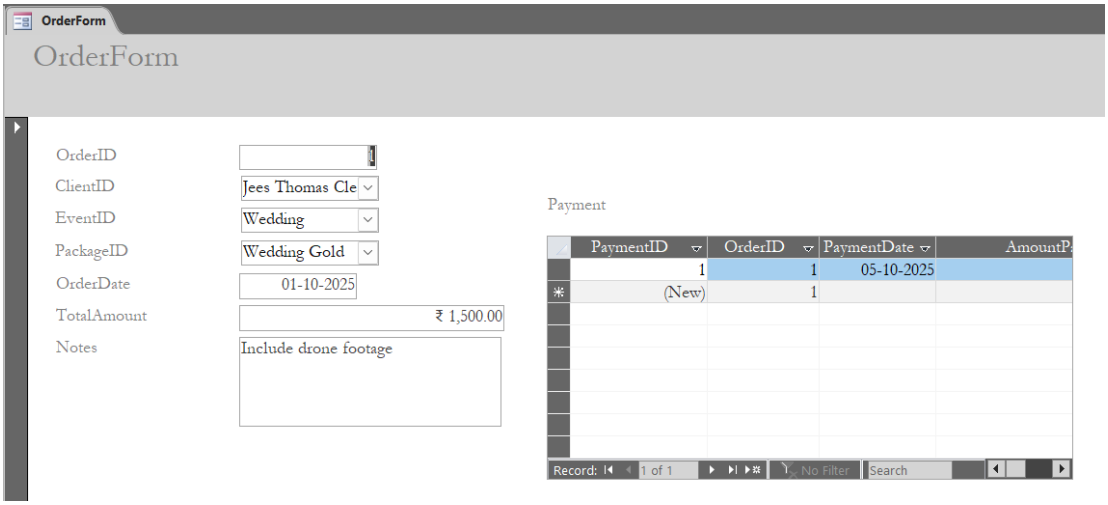
Figure 3.3: Package Form



Used to add or modify available photography packages and their pricing.

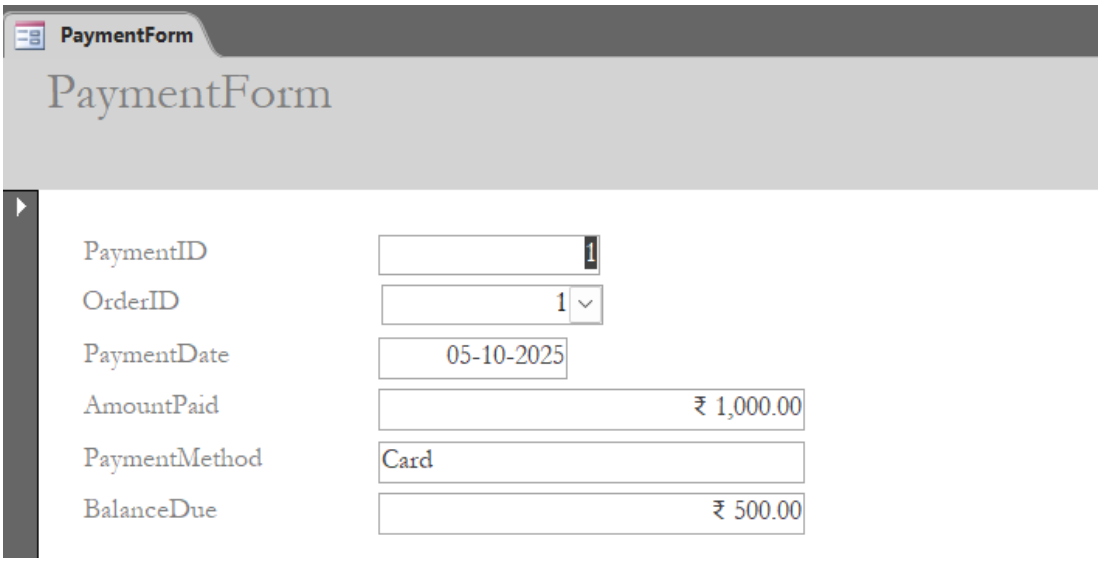
Helps in maintaining a clean record of services offered.

Figure 3.4: Order Form



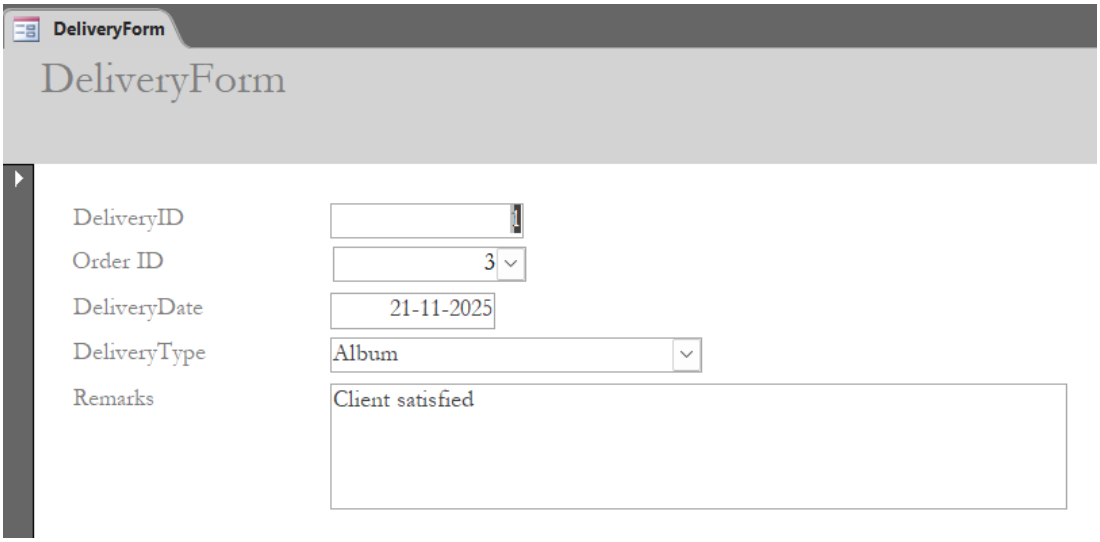
Used to create new client orders by linking client, event, and package information. Serves as the main operational form for managing photography bookings.

Figure 3.6: Payment Form



Used to record payments for each order and calculate pending balance automatically. Simplifies tracking of how much each client has paid.

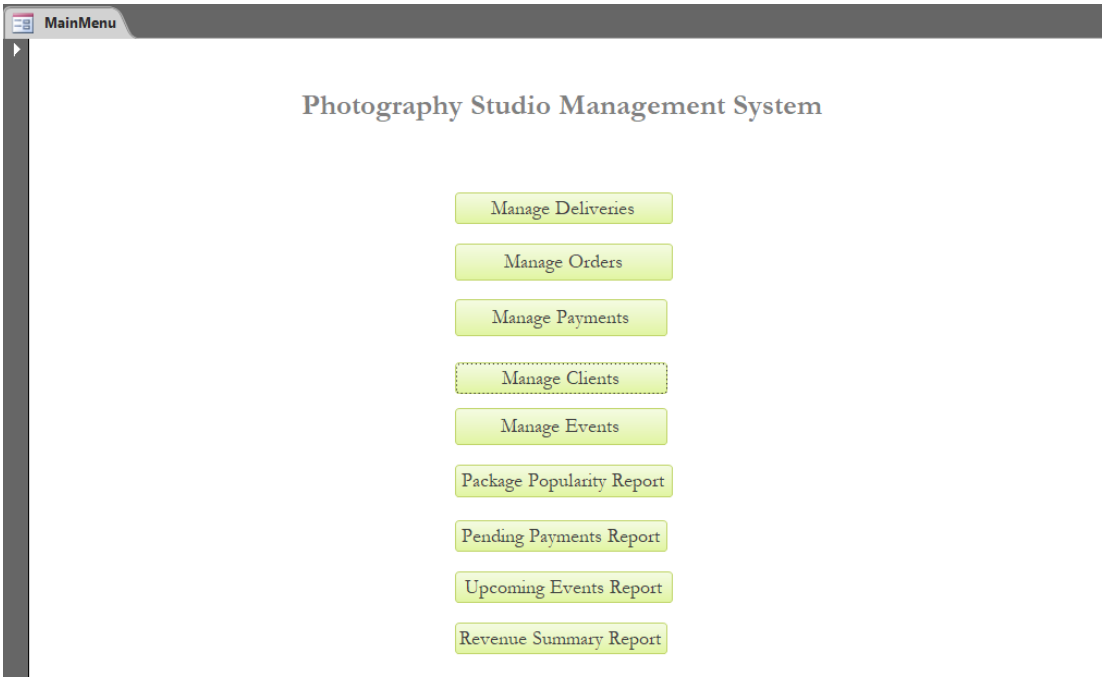
Figure 3.7: Delivery Form



Used to enter delivery details including type (USB, Album, Online) and remarks.

Helps maintain delivery records for completed photography projects.

Figure 3.8: Main Menu Form

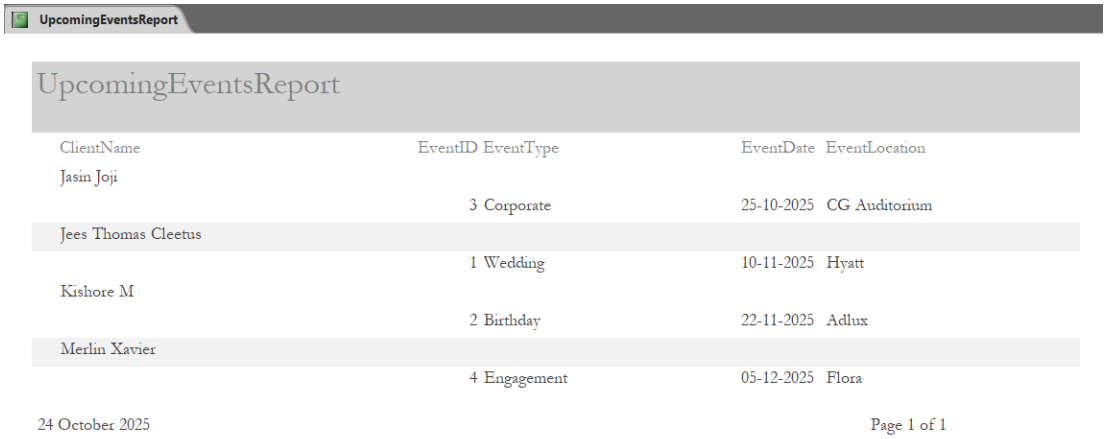


Acts as a user-friendly home screen for navigating between forms and reports.

Used for quick access to all database features from one dashboard.

4. Reports

Figure 4.1: Upcoming Events Report


UpcomingEventsReport

UpcomingEventsReport

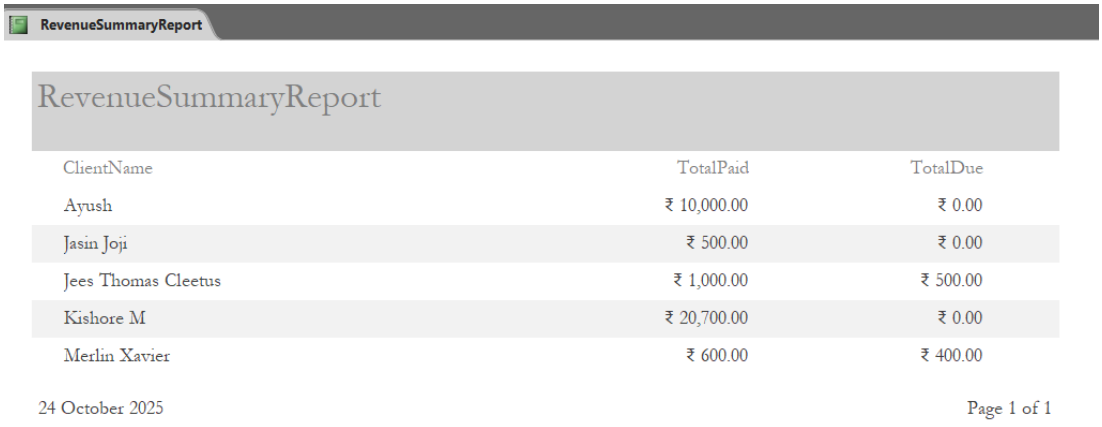
ClientName	EventID	EventType	EventDate	EventLocation
Jasin Joji				
	3	Corporate	25-10-2025	CG Auditorium
Jees Thomas Cleetus				
	1	Wedding	10-11-2025	Hyatt
Kishore M				
	2	Birthday	22-11-2025	Adlux
Merlin Xavier				
	4	Engagement	05-12-2025	Flora

24 October 2025 Page 1 of 1

Shows all upcoming events with date, client, and type in a formatted layout.

Used for scheduling and managing the studio's future assignments.

Figure 4.2: Revenue Summary Report


RevenueSummaryReport

RevenueSummaryReport

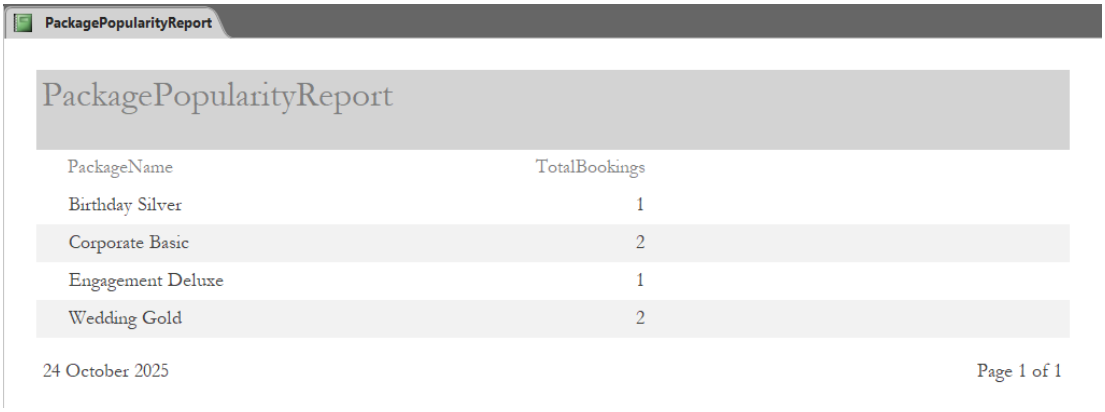
ClientName	TotalPaid	TotalDue
Ayush	₹ 10,000.00	₹ 0.00
Jasin Joji	₹ 500.00	₹ 0.00
Jees Thomas Cleetus	₹ 1,000.00	₹ 500.00
Kishore M	₹ 20,700.00	₹ 0.00
Merlin Xavier	₹ 600.00	₹ 400.00

24 October 2025 Page 1 of 1

Displays total amount received and pending payments per client.

Used for analyzing business income and outstanding dues.

Figure 4.3: Package Popularity Report


PackagePopularityReport

PackagePopularityReport

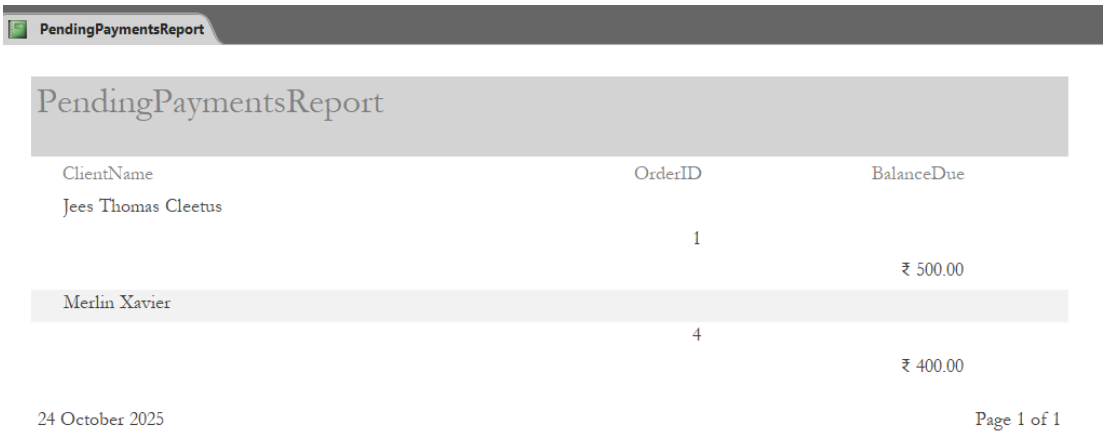
PackageName	TotalBookings
Birthday Silver	1
Corporate Basic	2
Engagement Deluxe	1
Wedding Gold	2

24 October 2025 Page 1 of 1

Shows which packages are most frequently ordered by clients.

Used for identifying popular services and improving marketing focus.

Figure 4.5: Pending Payments Report


PendingPaymentsReport

PendingPaymentsReport

ClientName	OrderID	BalanceDue
Jees Thomas Cleetus		
	1	
		₹ 500.00
Merlin Xavier		
	4	
		₹ 400.00

24 October 2025 Page 1 of 1

Shows clients with unpaid balances and due orders in a clear printable list.

Used for follow-up and tracking incomplete payment records.